

PythonLife

Basic to Advance Python Programming Core Python

Basics For Programming:

1. What is Programming?
2. What is Coding?
3. Compiler
4. Internal Structure of Compiler
5. Interpreter
6. Compiler vs Interpreter
7. Byte code
8. High Level Language
9. What is Low Level Language

What is Programming?

Programming is the process of creating a set of instructions that tell a computer how to perform a task.

Programming can be done using a variety of computer programming languages such as JavaScript, Python, and C++.

What is Coding?

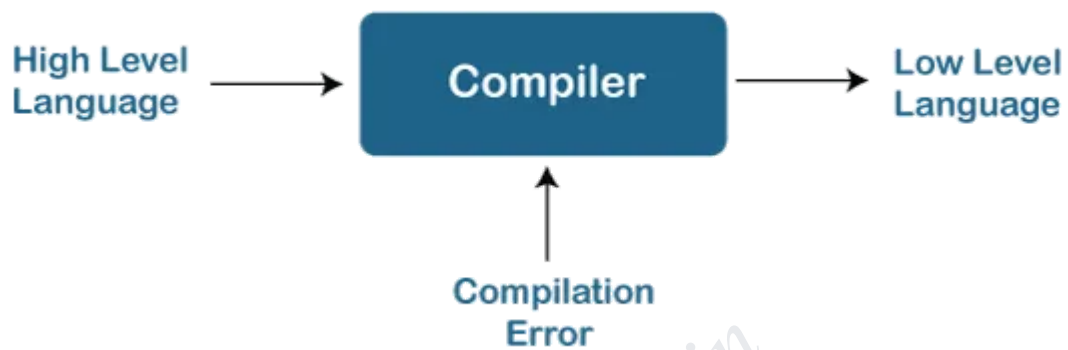
Coding is a list of step by step instructions that get computers to do what you want them to do.

Coding makes it possible for us to create computer software, games, apps and websites. Coders or Programmers behind everything we see and do on a computer.

Compiler:

The language processor that reads the complete source program written in high-level language as a whole in one go and translates it into an equivalent program in machine language is called a Compiler.

Example: C, C++, C#, Java.



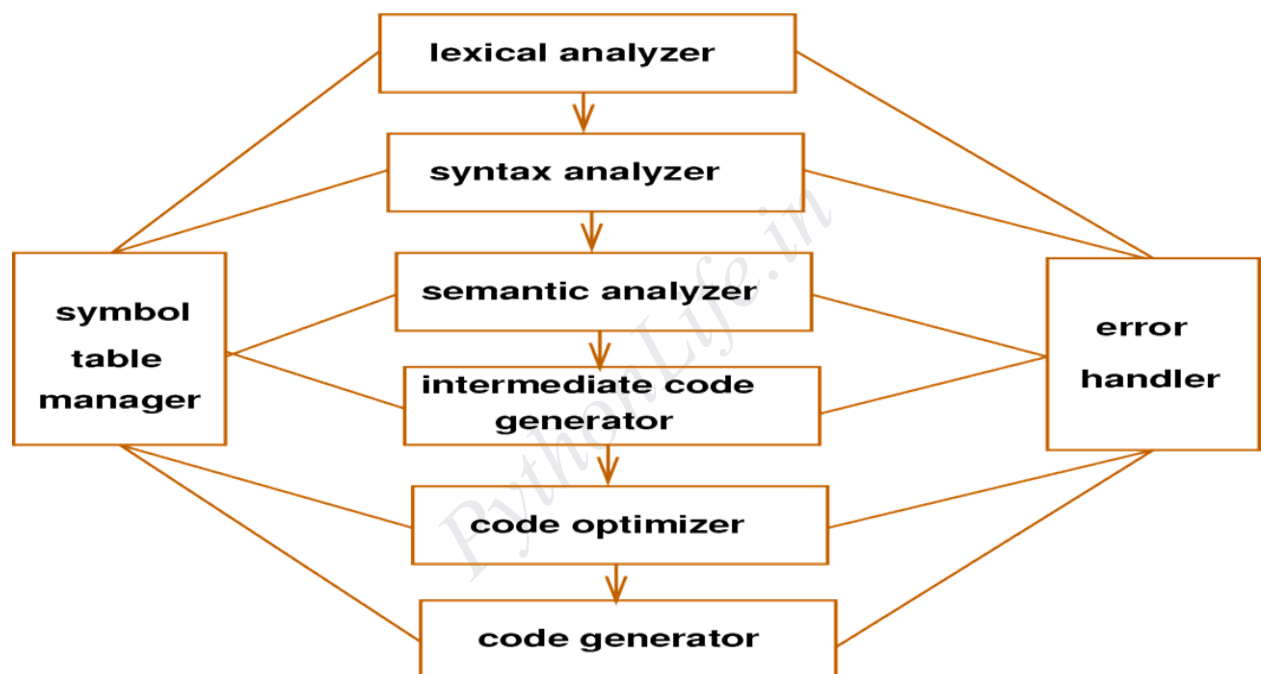
Internal Structure of Compiler:

Compiler operates in various phases each phase transforms the source program from one representation to another.

Every phase takes inputs from its previous stage and feeds its output to the next phase of the compiler.

There are 6 phases in a compiler. Each of this phase help in converting the high-level language the machine code.

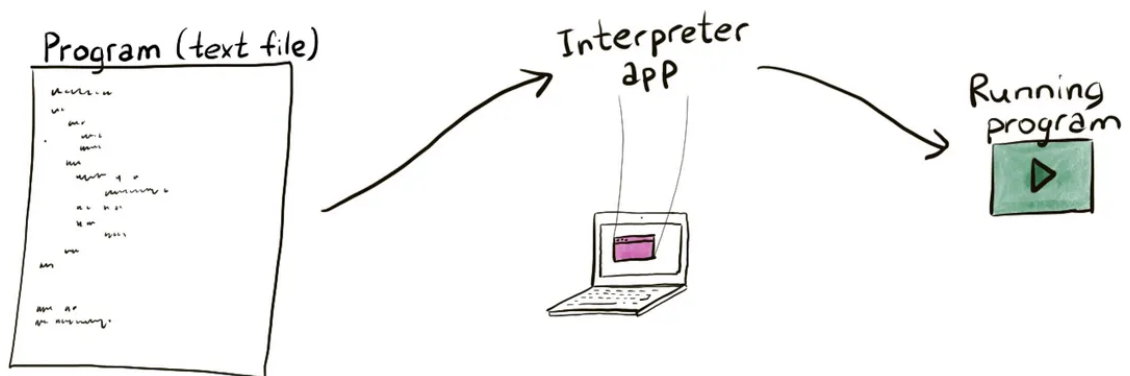
The phases of a compiler are: Lexical analysis Syntax analysis Semantic analysis Intermediate code generator Code optimizer, Code generator



Interpreter:

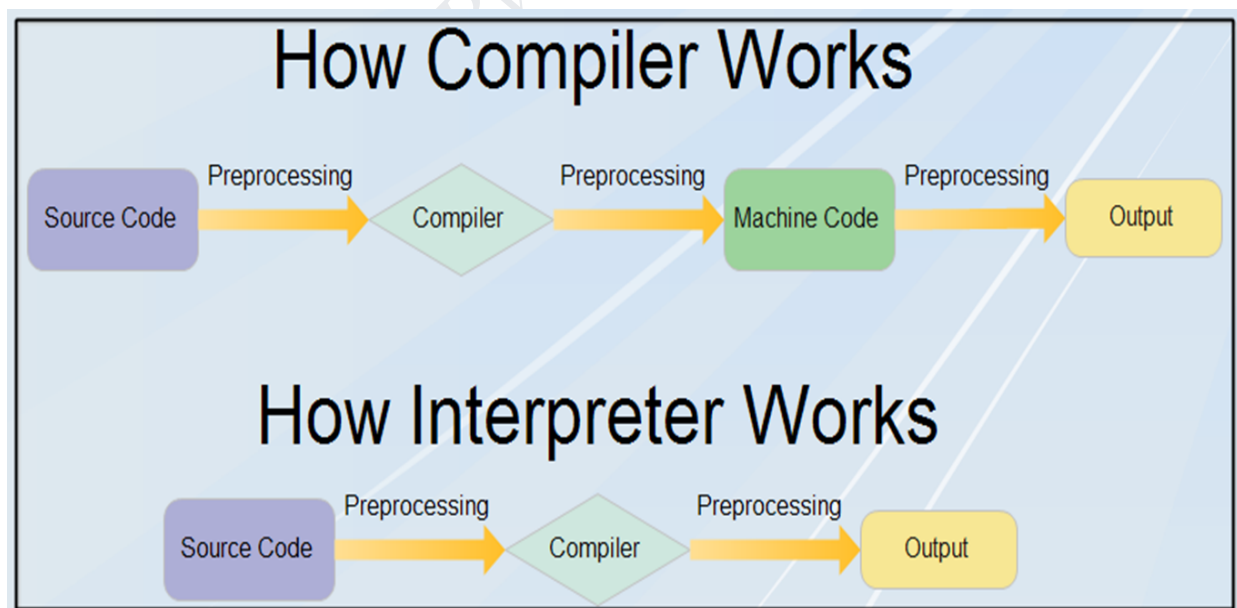
The Python interpreter is a virtual machine, meaning that it is software that emulates a physical computer.

This particular virtual machine is a stackmachine: it manipulates several stacks perform its operations (as contrasted with a register machine, which writes to and reads from particular memory locations).



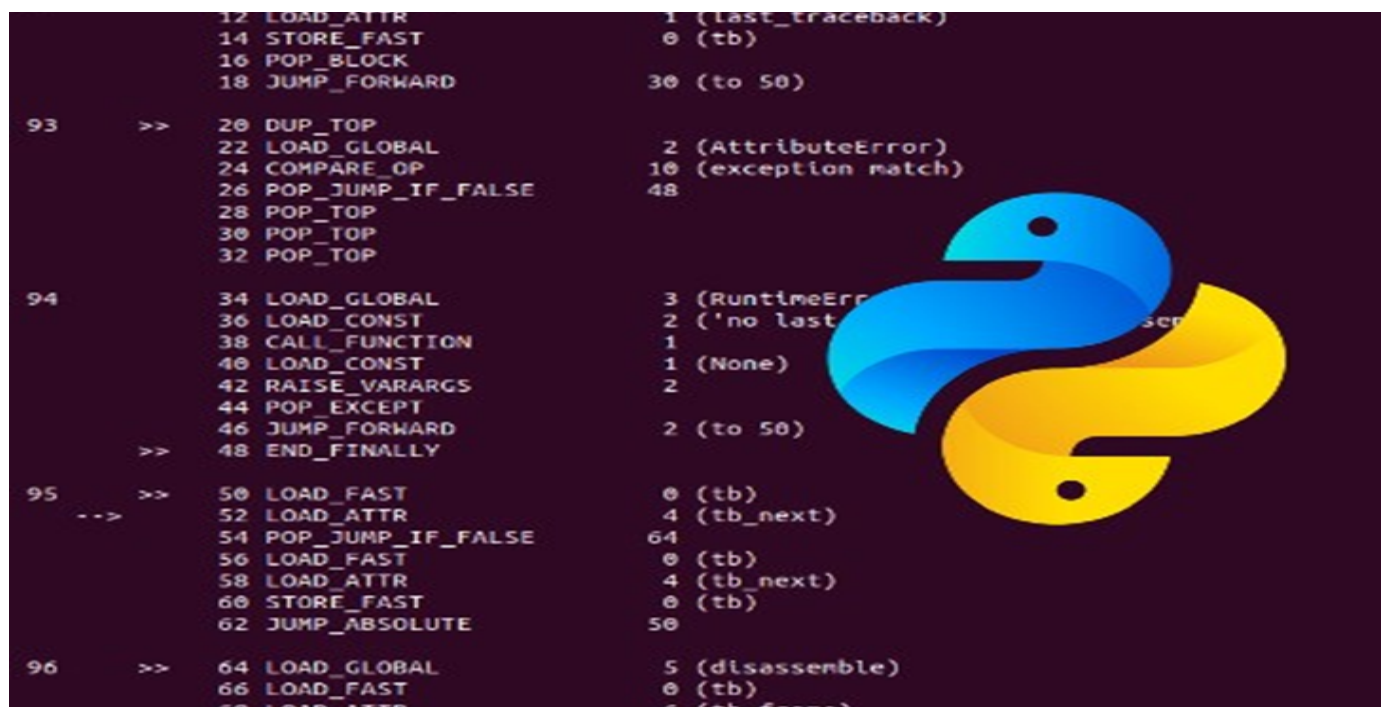
Compiler vs Interpreter:

Compiler	Interpreter
Compiler scans the entire program and translates the whole of it into machine code at once.	Interpreter translates just one statement of the program at a time into machine code
A compiler takes a lot of time to analyze the source code. However, the overall time taken to execute the process is much faster.	An interpreter takes very less time to analyze the source code. However, the overall time to execute the process is much slower.
A compiler always generates an intermediary object code. It will need further linking. Hence more memory is needed.	An interpreter does not generate an intermediary code. Hence, an interpreter is highly efficient in terms of its memory.
A compiler generates the error message only after it scans the complete program and hence debugging is relatively harder while working with a compiler. It is used in languages like C and C++ for example.	Keeps translating the program continuously till the first error is confronted. If any error is spotted, it stops working and hence debugging becomes easy. It is used in languages like Ruby and Python for example.



Byte code:

The byte code is a low-level platform independent representation of your source code. However, it is not the binary machine code and cannot be run by the target machine directly. In fact, it is a set of instructions for a virtual machine which is called the Python Virtual Machine (PVM).



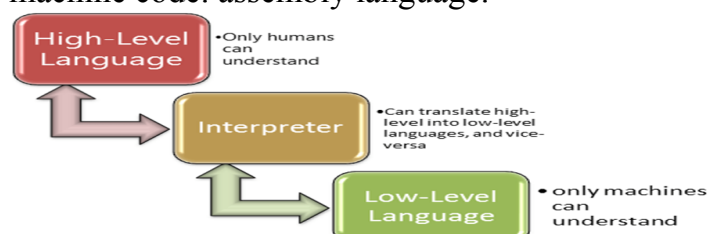
High Level Language:

A high-level is any programming language that enables development of a program in a much more user-friendly programming context and is generally independent of a computer's hardware architecture.

Low Level Language:

Low-level languages are languages that sit close to the computer's instruction set. An instruction set is the set of instructions that the processor understands.

Two types of low-level language are: machine code, assembly language.



Introduction to Python

- **What is Python?**
- **Why Python?**
- **History**
- **Features – Dynamic, Interpreted, Object oriented, Embeddable, Extensible, Large standard libraries, Free and Open source**
- **What is PVM?**
- **Python applications**
- **Python versions**
- **Python in real time industry**
- **Flavors of Python.**
- **Different Domains After Python.**
- **Python Module**
- **Python Package**
- **Why Python is More Popular**
- **Companies Offering Python Jobs**
- **Job Roles in Python**
- **Python Library**
- **Python Framework**
- **After Python**
- **Python Syntax compared to other programming languages**

What is Python ?

- Python is an interpreted, object-oriented, high-level programming language with dynamic semantics.
- Its high-level built in data structures, combined with dynamic typing and dynamic binding, make it very attractive for Rapid Application Development, as well as for use as a scripting or glue language to connect existing components together.

- Python's simple, easy to learn syntax emphasizes readability and therefore reduces the cost of program maintenance.
- Python supports modules and packages, which encourages program modularity and code reuse.

Why Python?

REASONS WHY PYTHON IS ONE OF THE BEST PROGRAMMING LANGUAGES:

- Python is Easy to Learn and Use
- Python is Handy for Web Development Purposes
- The Language is Extensively used in Data Science
- Has Multiple Libraries and Frameworks
- Python can be used in ML tool
- Flexibility and Reliability
- Python Automates Tasks

PythonLife.in

Python History:

- Python laid its foundation in the late 1980s.
- The implementation of Python was started in December 1989 by Guido Van Rossum at CWI in Netherland.
- In February 1991, Guido Van Rossum published the code labeled version 0.9.0 to alt.sources.
- In 1994, Python 1.0 was released with new features like lambda, map, filter, and reduce.
- Python 2.0 added new features such as list comprehensions, garbage collection systems.
- On December 3, 2008, Python 3.0 (also called "Py3K") was released. It was designed to rectify the fundamental flaw of the language.
- ABC programming language is said to be the predecessor of Python language, which was capable of Exception Handling and interfacing with the Amoeba Operating System.
- The following programming languages influence Python:
1.ABC language 2.Modula-3

Features of Python:

- Python is a general-purpose high-level programming language whose design philosophy emphasizes code readability.
- Python supports multiple programming paradigms, primarily but not limited to object oriented, imperative and, to a lesser extent, functional programming styles.
- Python aims to combine "remarkable power with very clear syntax", and its standard library is large and comprehensive.
- Its use of indentation for block delimiters is unusual among popular programming languages.

Features in Python :

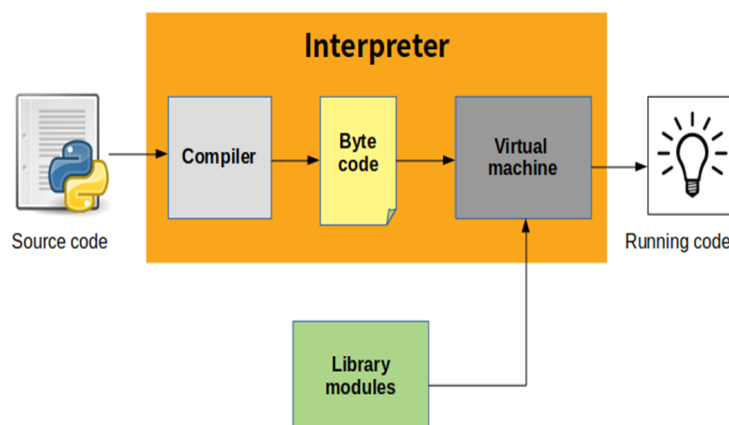
There are many features in Python, some of which are discussed below as follows:

- Free and Open Source
- Easy to code
- Easy to Read
- Object-Oriented Language
- GUI Programming Support
- High-Level Language
- Extensible feature
- Easy to Debug
- Python is a Portable language
- Python is an Integrated language
- Interpreted Language
- Large Standard Library
- Dynamically Typed Language

What is PVM?

Python Virtual Machine (PVM) is a program which provides programming environment. The role of PVM is to convert the byte code instructions into machine code so the computer can execute those machine code instructions and display the output.

Interpreter converts the byte code into machine code and sends that machine code to the computer processor for execution.



Python applications :

1. Web Applications
2. Enterprise Applications
3. Software Development Applications
4. Desktop GUI Applications
5. 3D CAD Applications
6. Scientific and Numeric Applications
7. Image Processing Applications
8. Audio or Video-based Applications
9. Console-based Applications
10. Business Applications



Python versions:

Python version hon Ve	RReleasedDate eleased
Python 1.0	January 1994
Python 1.5	December 31, 1997
Python 1.6	September 5, 2000

Python 2.0	October 16, 2000
Python 2.1	April 17, 2001
Python 2.2	December 21, 2001
Python 2.3	July 29, 2003
Python 2.4	November 30, 2004
Python 2.5	September 19, 2006
Python 2.6	October 1, 2008
Python 2.7	July 3, 2010
Python 3.0	December 3, 2008
Python 3.1	June 27, 2009
Python version	Released date
Python 3.2	February 20, 2011
Python 3.3	September 29, 2012
Python 3.4	March 16, 2014
Python 3.5	September 13, 2015
Python 3.6	December 23, 2016
Python 3.7	June 27, 2018
Python 3.8	October 14, 2019

Flavors of Python

Flavors of Python simply refers to the different Python compilers. These flavors are useful to integrate various programming languages into Python. Let us look at some of these flavors :

1. **CPython** : CPython is the Python compiler implemented in C programming language. In this, Python code is internally converted into the **byte code** using standard C functions. Additionally, it is possible to run and execute programs written in C/C++ using CPython compiler.
2. **Jython** : Earlier known as **JPython**. Jython is an implementation of the Python programming language designed to run on the Java platform. Jython is extremely useful because it provides the productivity features of a mature scripting language while running on a JVM.
3. **PyPy** : This is the implementation using Python language. PyPy often runs faster than CPython because PyPy is a just-in-time compiler while CPython is an interpreter.
4. **IronPython** : IronPython is an open-source implementation of the Python programming language which is tightly integrated with the .NET Framework.
5. **RubyPython** : RubyPython is a bridge between the Ruby and Python interpreters. It embeds a Python interpreter in the Ruby application's process using FFI (Foreign Function Interface).
6. **Pythonxy** : Python(x,y) is a free scientific and engineering development software for numerical computations, data analysis and data visualization based on Python.
7. **StacklessPython** : Stackless Python is a Python programming language interpreter. In practice, Stackless Python uses the C stack, but the stack is cleared between function calls.
8. **AnacondaPython** : Anaconda is a free and open-source distribution of the Python and R programming languages for scientific computing, that aims to simplify package management and deployment. Package versions are managed by the package management system conda.

Different Domains After Python:

- Web Development
- Testing
- Web Scraping
- Data Analysis
- Computer Graphics
- Machine Learning
- Big Data
- Internet of Things
- Data Science
- Deep Learning
- Artificial intelligence

Python Module :

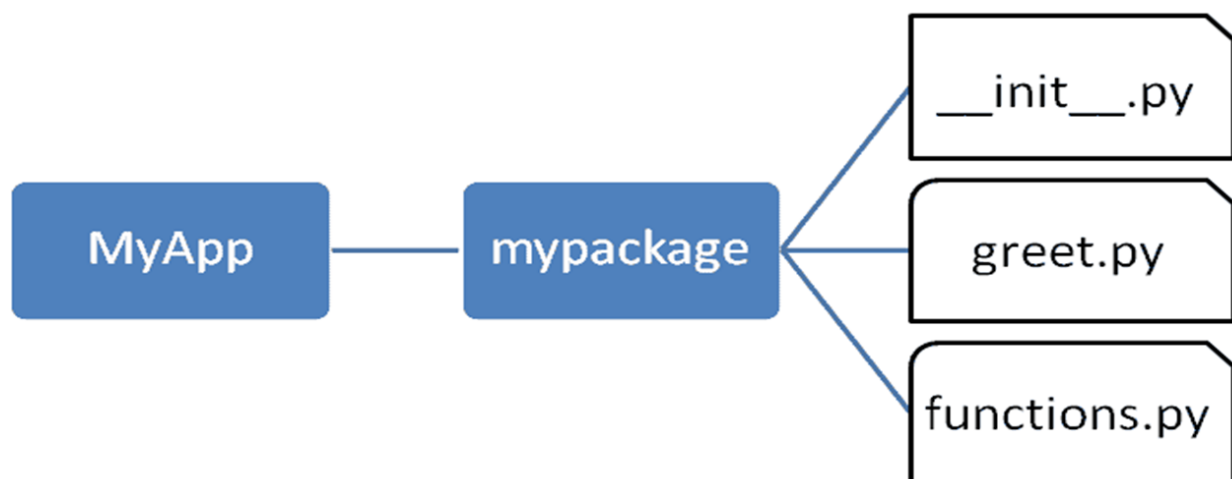
1. Python modules are nothing but files that consist of different statements and functions defined inside.
2. A module can define functions, classes, and variables. Modules help in organizing the code making it easier to use and understand. Modules provide reusability of the code.
3. Any file with extension .py can be referred as a module and the functions defined inside the module can be used in another program by simply using the import statement.
4. Suppose we need a function to find factorial in many programs. So, instead of defining a function to find the factorial in each program, what we can do is create a module with a function to find factorial and use that function in every program by simply importing the module.

Python Package:

A python package creates a hierarchical directory structure with numerous modules and sub-packages to give an application development environment. They are simply a collection of modules and sub-packages.

Importance of Python Packages :

- When working on a large or complex project, we frequently wind up with multiple modules. Using, maintaining, and organising so many modules is challenging.
- Fortunately, Python delivers us a simple solution in the form of packages. Using packages, we can easily group, organise, and use multiple modules.
- Packages also allow us to access all functions from all modules in a package with just one import statement.



Why Python is More Popular:

- Python is transparent.
- Python is open source.
- Python provides great support for developers.
- Python has a style guide for its code.
- Python reduces development time.
- Easy to learn and use.
- Supports multiple programming paradigms.
- Hundreds of Python and Frameworks Available Online.
- Growing Market Demand.



Companies Offering Python Jobs :

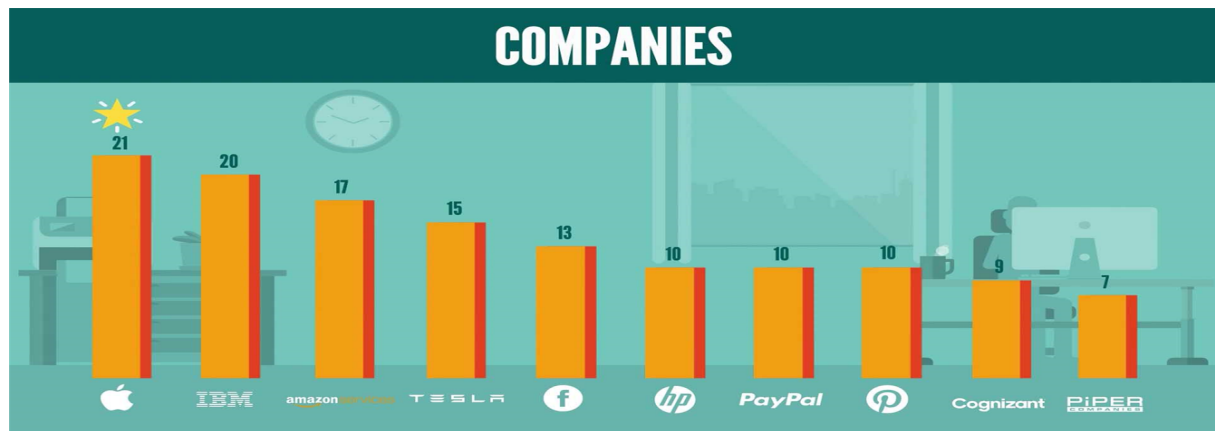
Python is a popular programming language among both smaller and larger companies – below are a few of the many big names which use the Python language to develop their projects.

Google – Python is one of the few official Google server-side languages. The plan for Google's backend was to use Python whenever possible. Python's main aim was to ensure undemanding maintenance and fast delivery.

Spotify – this music streaming platform uses Python for data analysis and backend. Python was chosen because it accelerates the development process. The coding in Python makes the development pipeline faster.

Reddit – half a year after deployment, the whole site was rewritten into Python from the Lisp language. In this case, the variety of libraries is as important as the readability of the

code.



Job Roles in Python:

1. Software Engineer
2. Web Developer/Front-end Developer
3. Python Developer
4. DevOps Engineer
5. Research Analyst
6. Data Analyst
7. Consultant
8. Product Manager
9. Data Scientist
10. Machine Learning Engineer

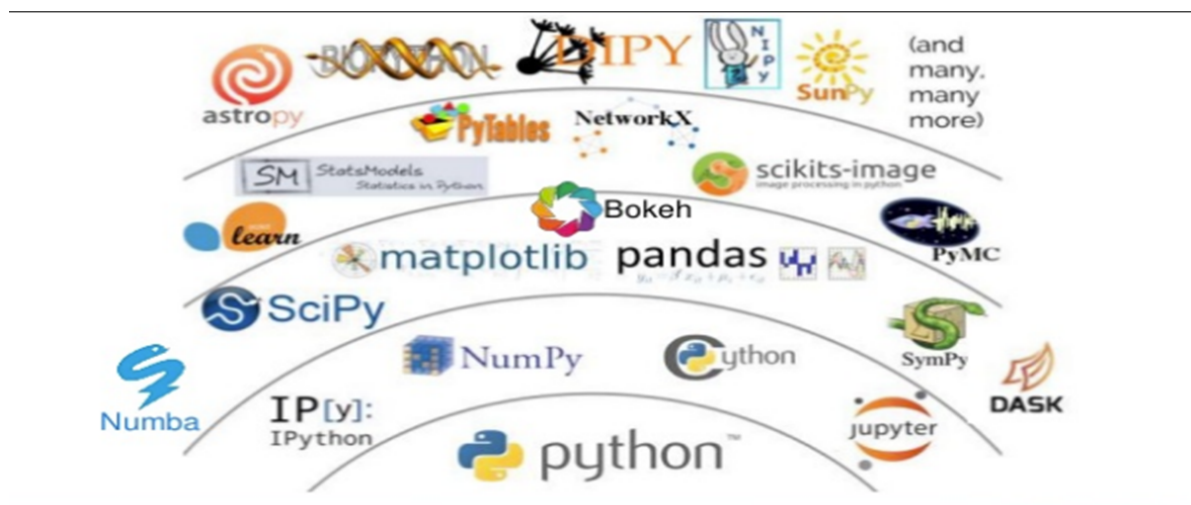
Python Library :

A Python library is a collection of related modules.

It contains bundles of code that can be used repeatedly in different programs.

It makes Python Programming simpler and convenient for the programmer.

As we don't need to write the same code again and again for different programs.



Important Python Libraries:

- TensorFlow
- Scikit-Learn
- Numpy
- Keras
- PyTorch
- LightGBM
- Eli5
- SciPy
- Theano
- Pandas
- Matplotlib
- Requests
- SQLAlchemy
- Pyglet

Python Frameworks:

Python is one of the most acceptable languages among web and application developers because of its strong emphasis on efficiency and readability.

There are numerous outstanding Python web frameworks, each with their own specialities and features.

Types of Python Framework:

Python majorly has three categories of frameworks and those are full-stack framework, micro-framework, and asynchronous framework. Now, let's understand what each category offers:

1. **Full Stack Framework:** As the name suggests this kind of framework provides a complete solution for web development like form generator, form validation, template layouts, etc. This type of framework can be utilized for any type of application. It is a little bit complex to use.
2. **Micro Framework:** It's a lightweight, easy-to-use framework and doesn't provide any extra features like data abstraction layer, form validation, etc. Developers have to put in lots of effort in adding code manually to get additional features and functionalities. This is useful for small applications.
3. **Asynchronous Framework:** This framework is gaining popularity in recent times and it uses asyncio library to work. This kind of framework mainly helps in running concurrent connections in huge amounts.

There are various frameworks of python like:

1. Bottle
2. Flask
3. Django
4. Web2py
5. AIOHTTP
6. CherryPy
7. Dash
8. Falcon
9. Growler
10. UvLoop
11. Pyramid
12. Sanic
13. CubicWeb

- 14. TurboGears
- 15. Hug
- 16. MorePath

After Python :

- 1. Teach Python to Beginners
- 2. Web Development with Python
- 3. Quality Assurance Engineer
- 4. Python Full Stack Developer
- 5. GIS Analyst
- 6. Data Scientist
- 7. Machine Learning Engineer
- 8. DevOps Engineer
- 9. Game Developer

Python Syntax compared to other programming languages :

Python vs PHP

	Python	php
Popularity	Very popular	Very popular
Frameworks	A lot of frameworks	A few frameworks
Learning	Easy to learn	Harder to learn

Python vs Java

	python	java
Learning	Easy to learn	Harder to learn
Cross-platform apps	Possible	Not Possible
Compatibility with operating systems	Possible	Possible
Network based apps	Not Possible	Possible

Python vs C#

	Python	C#
Simplicity	Possible	Not Possible
Script writing	In any environment	Only in IDE
Libraries	A lot of libraries	Few libraries
Performance	Low	High

Python vs Ruby

	python	Ruby
Approach to a problem	One solution	A lot of solutions
Community	Large	Large
Syntax	Very simple	More complex

Package After The Python Course
4LPA (Fresher)

PythonLife.in

The print statement:

- The print() function prints the given object to the standard output device (screen) or to the text stream file.
- Remember the time when you wrote your first program? Yes! I am talking about the “Hello World” program, which is probably the first program that anyone learns in their life.

Comments :

Comments in Python are identified with a hash symbol, #, and extend to the end of the line. Hash characters in a string are not considered comments, however. There are three ways to write a comment - as a separate line, beside the corresponding statement of code, or as a multi-line comment block.

There are multiple uses of writing comments in Python. Some significant uses include:

- Increasing readability
- Explaining the code to others
- Understanding the code easily after a long-term
- Including resources
- Re-using the existing code

Different Types of Comments:

There are three types of comments: single-line, multi-line. The syntax of comments varies depending on the type. This tutorial will explore every kind of comment individually, along with examples.

Single-Line Comments:

Single-line comments begin with the “#” character. Anything that is written in a single line after ‘#’ is considered as a comment. The syntax for writing single-line comments is:

```
# comments here
```

There are two ways of using single-line comments in Python. You can use it before the code or next to the code. The example depicted below shows the use of comments in both ways.

Data Types:

Data types are the classification or categorization of data items. It represents the kind of value that tells what operations can be performed on a particular data. Since everything is an object in Python programming, data types are actually classes and variables are instance (object) of these classes.

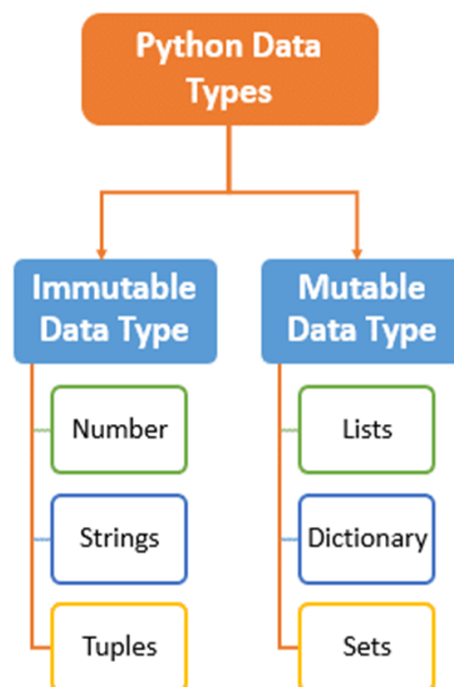
Core Data Types in Python :

As the name suggests, a data type is the classification of the type of values that can be assigned to variables. We have already seen that, here in Python, we don't need to declare a variable with explicitly mentioning the data type, but it's still important to understand the different types of values that can be assigned to variables in Python.

After all the data type of a variable is decided based on the value assigned.

in

python:



Let's discuss the above-mentioned core data types in Python.

Numbers: The number data type in Python is used to store numerical values. It is used to carry out normal mathematical operations.

Strings: Strings in Python are used to store textual information. They are used to carry out operations that perform positional ordering among items.

Lists: The list data type is the most generic Python data type. Lists can consist of a collection of mixed data types, stored by relative positions.

Tuples: Python Tuples are one among the immutable Python data types that can store values of mixed data types. They are basically a list that cannot be changed.

Sets: Sets in Python are a data type that can be considered as an unordered collection of data without any duplicate items.

Dictionaries: Dictionaries in Python can store multiple objects, but unlike lists, in dictionaries, the objects are stored by keys and not by positions.

Python Keywords :

Python keywords are reserved words. They are used by python interpreters to understand the program. Keywords define the structure of programs. We can't use keywords to name program entities such as variables, classes, and functions.

and	else	in	return
as	except	is	True
assert	finally	lambda	try
break	false	nonlocal	with
class	for	None	while
continue	from	not	yield
def	global	or	
del	if	pass	
elif	import	raise	

Identifier :

- Identifier is a name used to identify a variable, function, class, module, etc. The identifier is a combination of character digits and underscore.
- The identifier should start with a character or Underscore then use a digit.
- The characters are A-Z or a-z, an Underscore (_) , and digit (0-9).
- We should not use special characters (#, @, \$, %, !) in identifiers.

Examples for valid identifiers:

• `var1`

• `_var1`

• `_1_var`

• `var_1`

Examples for invalid identifiers:

• `!var1`

• `1var`

• `1_var`

• `var#1`

Variable:

- Python Variable is containers which store values.
- Python is not “statically typed”.
- We do not need to declare variables before using them or declare their type.
- A variable is created the moment we first assign a value to it.
- A Python variable is a name given to a memory location.
- It is the basic unit of storage in a program.

Rules for creating variables in Python:

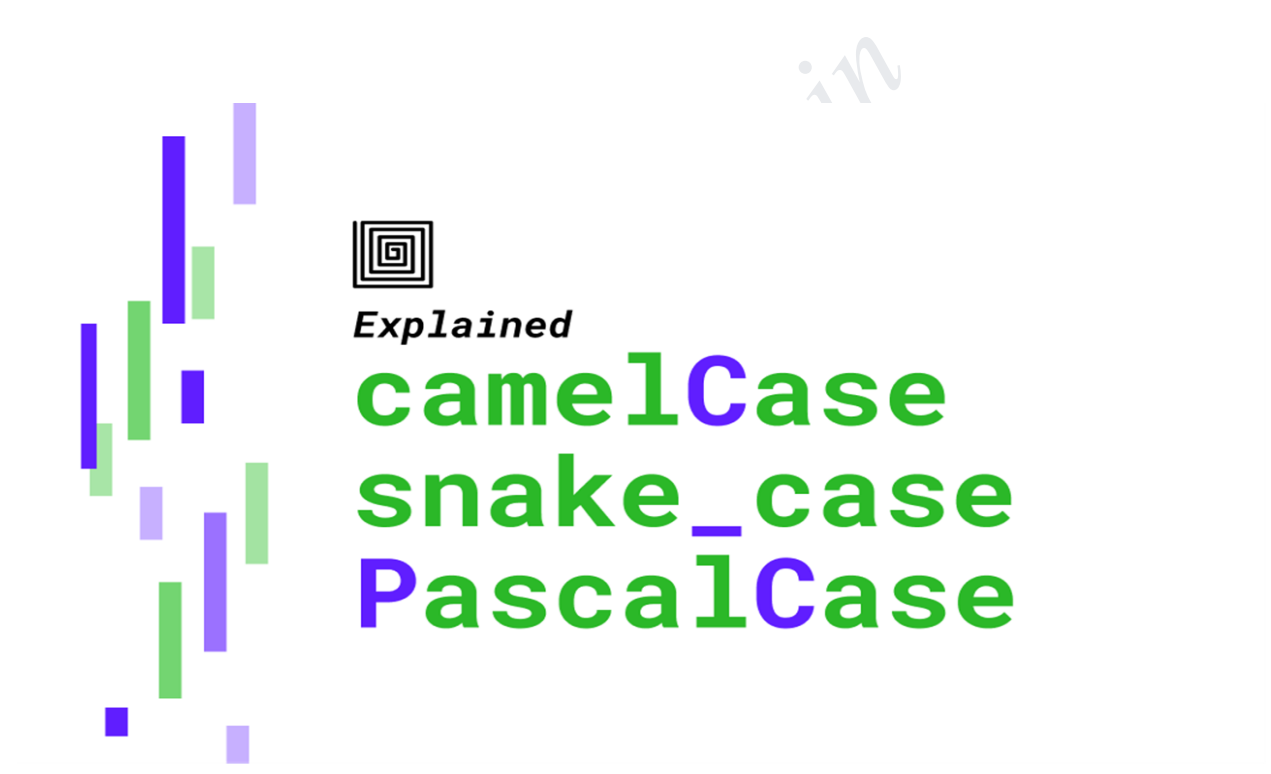
- A variable name must start with a letter or the underscore character.
- A variable name cannot start with a number.
- A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9, and _).
- Variable names are case-sensitive (name, Name and NAME are three different variables).
- The reserved words(keywords) cannot be used naming the variable.

Pythonlife.in

Variable Cases:

There are 3 types of variable cases in python:

1. Camel Case
2. Snake Case
3. Pascal Case



Camel Case:

- Suppose we have a list of words, we have to concatenate them in camel case format.
- So, if the input is like ["Hello", "World", "Python", "Programming"], then the output will be "helloWorldPythonProgramming".

ex: *someVar, someClass, somePackage.xyz.*

Snake Case:

Snake case (stylized as snake_case) refers to the style of writing in which each space is replaced by an underscore (_) character, and the first letter of each word is written in lowercase.

It is a commonly used naming convention in computing, for example for variable and subroutine names, and for filenames.

One study has found that readers can recognize snake case values more quickly than camel case.

ex: *some_var, some_class, some_package.xyz.*

Pascal Case:

Pascal case follows the same camel case naming convention rules — all but one: we capitalize the first letter.

In object-oriented languages like Java and TypeScript, we use pascal case to denote classes, namespaces, and abstractions like interfaces.

In scripting languages like Python, you'll rarely find camel or pascal case code except when defining a class's name.

ex: *SomeVar, SomeClass, SomePackage.xyz*

Data conversion functions – int(), float(), complex(), str()

int() -

- This function is used to convert any data type into an integer data type.
- The syntax of int() is int(variable, base), the function takes 2 parameters.
- where “variable” is a string and ‘base’ specifies the base in which string is if the data type is a string.

float() –

- This function is used to convert any data type into a float type.
- The syntax of float() is float(parameter), where parameter is optional.
- The use of float without parameters is only to declare an empty float data type variable.

complex()-

- The complex() function is used to create a complex number or convert a string or number to a complex number.
- The complex type is described in Numeric Types – int, float, complex.

str() –

- Used to convert an integer into a string.
- This function is mostly used when we need to combine different data type values with string data type values.
- Syntax of str() function is str(parameter).

Input():

- In Python, we use input() function to take input from the user.
- Whatever you enter as input, the input function converts it into a string.
- If you enter an integer value still input() function convert it into a string.

Syntax:

input(prompt)

Parameter:

Prompt: (optional) The string that is written to standard output (usually screen) without newline.

Return:

String object

Python Operators

- What is Operator?
- What is Operand?
- Arithmetic Operators
- Comparison Operators
- Python Assignment Operators
- Logical Operators
- Membership Operators
- Identity Operators

What is Operator?

- Python Operators in general are used to perform operations on values and variables.
- These are standard symbols used for the purpose of logical and arithmetic operations.
- Operators: Are the special symbols.

Eg- + , * , /, etc

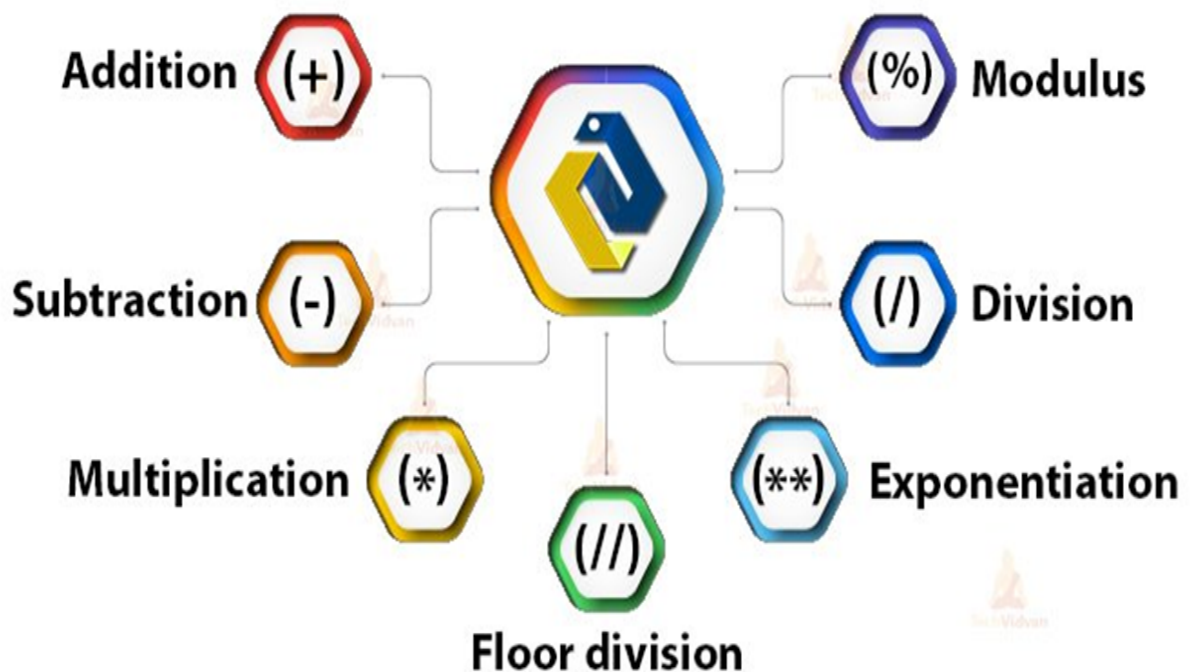
What is Operand?

The operands in python, one of the programming languages , refer to the values on which the operator operates.

Arithmetic Operators :

Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication, etc.

Python Arithmetic Operators



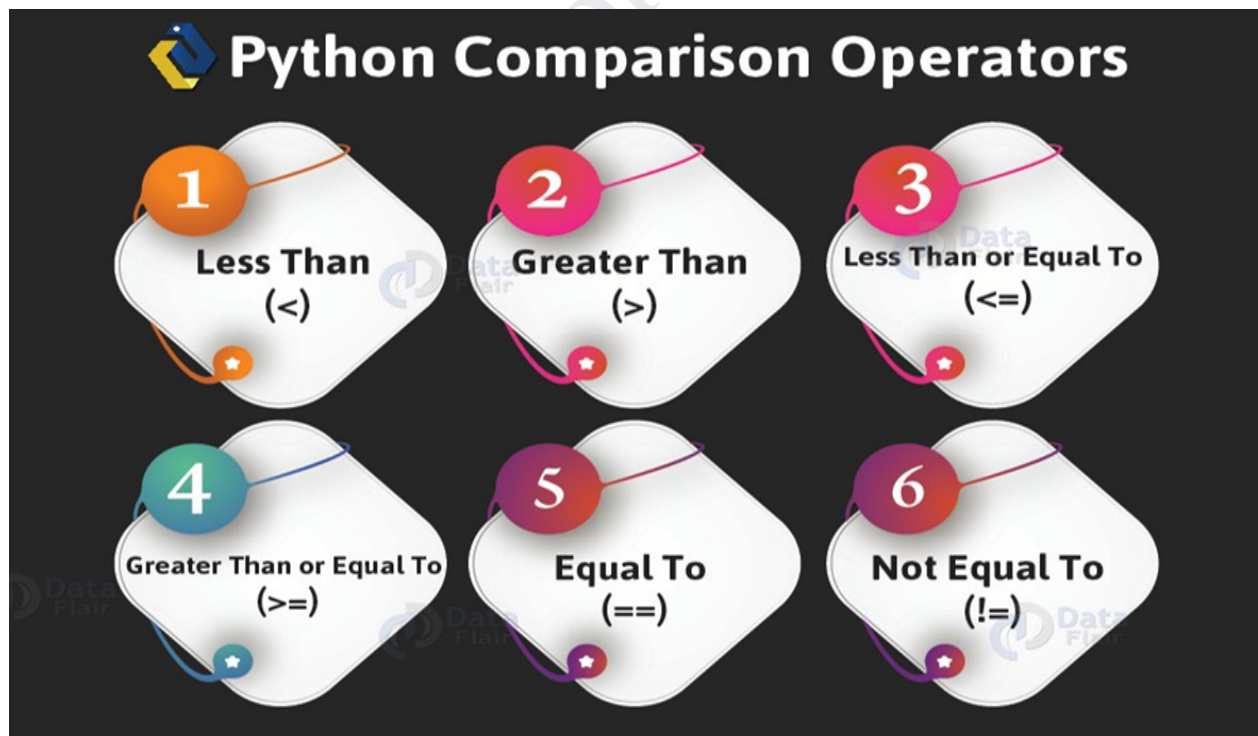
Operator	Meaning	Example
+	Add two operands or unary plus	$x + y$
-	Subtract right operand from the left or unary minus	$x - y$
*	Multiply two operands	$x * y$
/	Divide left operand by the right one (always results into float)	x/y
%	Modulus - remainder of the division of left operand by the right	$x\%y =$ (remainder of y/x)

//	Floor division - division that results into whole number adjusted to the left in the number line	$x // y$
**	Exponent - left operand raised to the power of right	$x ** y$ (x to the power y)

Comparison Operators:

In programming, you often want to compare a value with another value. To do that, you use comparison operators. It returns either true or false according to the condition.

Python has six comparison operators, which are as follows:



Operator	Meaning	Example
<	Less than - True if left operand is less than the right	x < y
>	Greater than - True if left operand is greater than the right	x > y
<=	Less than or equal to - True if left operand is less than or equal to the right	x <= y
>=	Greater than or equal to - True if left operand is greater than or equal to the right	x >= y
==	Equal to - True if both operands are equal	x == y
!=	Not equal to - True if operands are not equal	x != y

Python Assignment Operators

- Assignment operators are used in Python to assign values to variables.
- `a=5` is a simple assignment operator that assigns the value 5 on the right to the variable a on the left.
- There are various compound operators in Python like `a += 5` that adds to the variable and later assigns the same. It is equivalent to `a=a+5`.

Operator	Meaning	Example
=	Assign value of right side of expression to left side operand	x = y + z
+=	Add and Assign: Add right side operand with left side operand and then assign to left operand	a += b
-=	Subtract AND: Subtract right operand from left operand and then assign to left operand: True if both operands are equal	a -= b
*=	Multiply AND: Multiply right operand with left operand and then assign to left operand	a *= b
/=	Divide AND: Divide left operand with right operand and then assign to left operand	a/=b
%=	Modulus AND: Takes modulus using left and right operands and assign result to left operand	a%=b
//=	Divide(floor) AND: Divide left operand with right operand and then assign the value(floor) to left operand	a //= b
**=	Exponent AND: Calculate exponent(raise power) value using operands and assign value to left operand	a **= b
&=	Performs Bitwise AND on operands and assign value to left operand	a &= b
=	Performs Bitwise OR on operands and assign value to left operand	a = b
^=	Performs Bitwise XOR on operands and assign value to left operand	a ^= b
>>=	Performs Bitwise right shift on operands and assign value to left operand	a >>= b
<<=	Performs Bitwise left shift on operands and assign value to left operand	a <<= b

Logical Operators

In Python, Logical operators are used on conditional statements (either True or False).

They perform Logical AND, Logical OR and Logical NOT operations.

Operators	Description
&& (Logical AND)	This operator returns true if all relational statements combined with && are true, else it returns false.
(Logical OR)	This operator returns true if at least one of the relational statements combined with is true, else it returns false.
! (logical NOT)	It returns the inverse of the statement's result.
XOR(Logical XOR operator)	This operator returns true if either statement one is true or statement two is true but not both.
and (Logical AND)	This operator returns true if all relational statements combined with it are true, else it returns false.
or (Logical OR)	This operator returns true if atleast one of the relational statements combined with it is true, else it returns false.

Membership Operators

Membership operators are used to test if a sequence is presented in an object:

Operator	Description	Example
in	Returns True if a sequence with the specified value is present in the object	x in y
not in	Returns True if a sequence with the specified value is not present in the object	x not in y

Identity Operators

Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location:

Operator	Description	Example
is	Returns true if both variables are the same object	x is y
is not	Returns true if both variables are not the same object	x is not y

--	--	--

Control Statements

- Conditional control statements
 - If
 - If-else
 - If-elif-else
 - Nested-if
- Looping statements
 - for
 - while
 - Nested loops
- Break
- Continue
- Pass
- Return

Conditional control statements

In programming languages, most of the time in large projects we have to control the flow of execution of our program and we want to execute some set of statements only if the given condition is satisfied, and a different set of statements when it's not satisfied.

Conditional statements are also known as decision-making statements. We need to use these conditional statements to execute the specific block of code if the given condition is true or false.

In Python we can achieve decision making by using the following statements:

- if statements
- if-else statements
- elif statements
- Nested if and if-else statements

elif ladder

If

Python if statement is one of the most commonly used conditional statements in programming languages. It decides whether certain statements need to be executed or not.

It checks for a given condition, if the condition is true, then the set of code present inside the " if " block will be executed otherwise not.

The if condition evaluates a Boolean expression and executes the block of code only when the Boolean expression becomes TRUE.

Syntax:

```
if (EXPRESSION == TRUE):
```

```
    Block of code
```

```
Else:
```

```
    Block of code
```

Here, the condition will be evaluated to a Boolean expression (true or false).

If the condition is true, then the statement or program present inside the " if " block will be executed and if the condition is false, then the statements or program present inside the "else" block will be executed.

If-else

The statement itself says if a given condition is true then execute the statements present inside the “if block” and if the condition is false then execute the “else” block.

The “else” block will execute only when the condition becomes false. It is the block where you will perform some actions when the condition is not true.

if-else statement evaluates the Boolean expression. If the condition is TRUE then, the code present in the “ if “ block will be executed otherwise the code of the “else“ block will be executed

Syntax :

```
if (EXPRESSION == TRUE):
```

```
    Statement (Body of the block)
```

```
else:
```

```
    Statement (Body of the block)
```

Here, the condition will be evaluated to a Boolean expression (true or false).

If the condition is true then the statements or program present inside the “if” block will be executed and if the condition is false then the statements or program present inside the “else” block will be executed.

If-elif-else

In Python, we have one more conditional statement called “elif” statements.

“elif” statement is used to check multiple conditions only if the given condition is false.

It's similar to an “if-else” statement and the only difference is that in “else” we will not check the condition but in “elif” we will check the condition.

“elif” statements are similar to “if-else” statements but “elif” statements evaluate multiple conditions.

Syntax:

if (condition):

 #Set of statement to execute if condition is true

elif (condition):

 #Set of statements to be executed when if condition is false and elif condition is

true

else:

 #Set of statement to be executed when both if and elif conditions are false

Nested if-else

Nested “if-else” statements mean that an “if” statement or “if-else” statement is present inside another if or if-else block.

Python provides this feature as well, this in turn will help us to check multiple conditions in a given program.

An “if” statement is present inside another “if” statement which is present inside another “if” statements and so on.

Syntax:

if boolean_expression:

 if boolean_expression:

```
statement(s)
```

```
else:
```

```
    statement(s)
```

```
else:
```

```
    statement(s)
```

Looping statements

The following loops are available in Python to fulfil the looping needs. Python offers 3 choices for running the loops. The basic functionality of all the techniques is the same, although the syntax and the amount of time required for checking the condition differ.

We can run a single statement or set of statements repeatedly using a loop command.

Name of the loop	Loop Type & Description
While loop	Repeats a statement or group of statements while a given condition is TRUE. It tests the condition before executing the loop body.
For loop	This type of loop executes a code block multiple times and abbreviates the code that manages the loop variable.
Nested loops	We can iterate a loop inside another loop.

for

It has the ability to iterate over the items of any sequence, such as a list or a string

Syntax:

for iterating_var in sequence:

```
    statement(s)
```

If a sequence contains an expression list, it is evaluated first. Then, the first item in the sequence is assigned to the iterating variable `iterating_var`.

Next, the statements block is executed. Each item in the list is assigned to `iterating_var`, and the statement(s) block is executed until the entire sequence is exhausted.

while

A while loop statement in Python programming language repeatedly executes a target statement as long as a given condition is true.

Syntax

The syntax of a while loop in Python programming language is -

while expression:

statements(s)

- Here, statement(s) may be a single statement or a block of statements. The condition may be any expression, and true is any non-zero value. The loop iterates while the condition is true.
- When the condition becomes false, program control passes to the line immediately following the loop.
- In Python, all the statements indented by the same number of character spaces after a programming construct are considered to be part of a single block of code. Python uses indentation as its method of grouping statements.

Nested loops

Python programming language allows to use one loop inside another loop. Following section shows few examples to illustrate the concept.

Syntax:

for `iterating_var` in sequence:

for `iterating_var` in sequence:

```
statement(s)
```

```
statement(s)
```

The syntax for a **nested while loop** statement in Python programming language is as follows -

while expression:

 while expression:

 statement(s)

statement(s)

A final note on loop nesting is that you can put any type of loop inside of any other type of loop. For example a for loop can be inside a while loop or vice versa.

Break

The break statement is used to break out of a loop. It is used inside for and while loops to alter the normal behavior. The break will end the loop it is in and control flows to the statement immediately below the loop.

Syntax:

```
Loop{  
    Condition:  
        break  
}
```

Example:

```
# break Statement Example  
for i in range(1, 10):  
    if i == 4:  
        break  
    print(i)
```

Note: One to four numbers will execute in the above program

Continue

The continue keyword is used to end the current iteration in a for loop or a while loop and continues to the next iteration.

The continue statement is a temporary break in a loop.

Syntax

continue

Example:

```
# continue Statement Example
for i in range(1, 10):
    if i == 4:
        continue
    print(i)
```

Note: One to 10 numbers will execute and except 4th number in the above program

Pass

The pass statement in Python is used when a statement is required syntactically but you do not want any command or code to execute.

The pass statement is a *null* operation; nothing happens when it executes.

The pass is also useful in places where your code will eventually go, but has not been written yet.

Syntax:

```
pass
```

Example:

```
for letter in 'Python':
    if letter == 'h':
        pass
    print('This is pass block')
print ('Current Letter :', letter)
print( "Good bye!")
```

Python List

A list in Python is used to store the sequence of various types of data.

Python lists are mutable type its mean we can modify its element after it created.

However, Python consists of six data-types that are capable to store the sequences, but the most common and reliable type is the list.

A list can be defined as a collection of values or items of different types.

The items in the list are separated with the comma (,) and enclosed with the square brackets []

A list can be define as below

```
L1 = ["John", 102, "USA"]
```

```
L2 = [1, 2, 3, 4, 5, 6]
```

If we try to print the type of L1, L2, and L3 using type() function then it will come out to be a list.

```
print(type(L1))
```

```
print(type(L2))
```

Output:

```
<class 'list'>
```

```
<class 'list'>
```

Characteristics of Lists :

The list has the following characteristics:

- The lists are ordered.
- The element of the list can access by index.
- The lists are the mutable type.
- The lists are mutable types.
- A list can store the number of various elements.

List indexing and splitting

The indexing is processed in the same way as it happens with the strings. The elements of the list can be accessed by using the slice operator [].

The index starts from 0 and goes to length - 1. The first element of the list is stored at the 0th index, the second element of the list is stored at the 1st index, and soon.

List = [0, 1, 2, 3, 4, 5]

0	1	2	3	4	5
---	---	---	---	---	---

List[0] = 0

List[0:] = [0,1,2,3,4,5]

List[1] = 1

List[:] = [0,1,2,3,4,5]

List[2] = 2

List[2:4] = [2, 3]

List[3] = 3

List[1:3] = [1, 2]

List[4] = 4

List[:4] = [0, 1, 2, 3]

List[5] = 5

We can get the sub-list of the list using the following syntax.

`list_variable(start:stop:step)`

- The start denotes the starting index position of the list.
- The stop denotes the last index position of the list.
- The step is used to skip the nth element within a start:stop

Consider the following example:

```
list = [1,2,3,4,5,6,7]
```

```
print(list[0])
```

```
print(list[1])
```

```
print(list[2])
```

```
print(list[3])
```

```
# Slicing the elements
```

```
print(list[0:6])
```

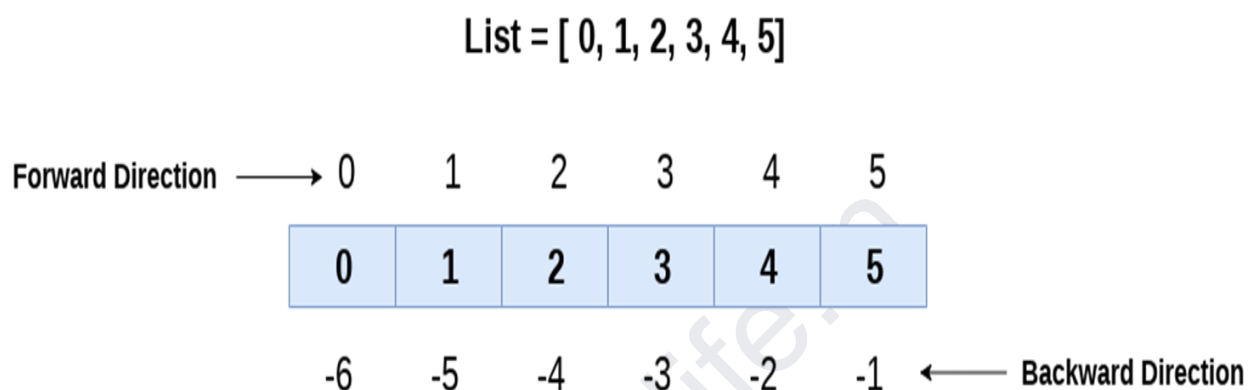
```
# By default the index value is 0 so its starts from the 0th element and go  
for index -1.
```

```
print(list[:])
```

```
print(list[2:5])
```

```
print(list[1:6:2])
```

Unlike other languages, Python provides the flexibility to use the negative indexing also. The negative indices are counted from the right. The last element (rightmost) of the list has the index -1; its adjacent left element is present at the index -2 and so on until the left-most elements are encountered.



Let's have a look at the following example where we will use negative indexing to access the elements of the list.

```
list = [1,2,3,4,5]
```

```
print(list[-1])
```

```
print(list[-3:])
```

```
print(list[:-1])
```

```
print(list[-3:-1])
```

As we discussed above, we can get an element by using negative indexing. In the above code, the first print statement returned the rightmost element of the list. The second print statement returned the sub-list, and so on.

Updating List values

Lists are the most versatile data structures in Python since they are mutable, and their values can be updated by using the slice and assignment operator.

Python also provides `append()` and `insert()` methods, which can be used to add values to the list.

Consider the following example to update the values inside the list

```
list = [1, 2, 3, 4, 5, 6]
```

```
print(list)
```

```
# It will assign value to the value to the second index
```

```
list[2] = 10
```

```
print(list)
```

```
# Adding multiple-element
```

```
list[1:3] = [89, 78]
```

```
print(list)
```

```
# It will add value at the end of the list
```

```
list[-1] = 25
```

```
print(list)
```

Iterating a List

A list can be iterated by using a for - in loop. A simple list containing four strings, which can be iterated as follows.

```
list = ["John", "David", "James", "Jonathan"]
```

```
for i in list:
```

```
    # The i variable will iterate over the elements of the List and contains  
    each element in each iteration.
```

```
    print(i)
```

Adding elements to the list

Python provides append() function which is used to add an element to the list. However, the append() function can only add value to the end of the list.

Consider the following example in which, we are taking the elements of the list from the user and printing the list on the console.

```
#Declaring the empty list
```

```
l = []
```

```
#Number of elements will be entered by the user
```

```
n = int(input("Enter the number of elements in the list:"))
```

```
# for loop to take the input
```

```
for i in range(0,n):
```

```
    # The input is taken from the user and added to the list as the item
```

```
    l.append(input("Enter the item:"))
```

```
print("printing the list items..")
```

```
# traversal loop to print the list items
```

```
for i in l:
```

```
print(i, end = " ")
```

Removing elements from the list

Python provides the `remove()` function which is used to remove the element from the list. Consider the following example to understand this concept.

Example -

```
list = [0,1,2,3,4]
print("printing original list: ");
for i in list:
    print(i,end=" ")
list.remove(2)
print("\nprinting the list after the removal of first element...")
for i in list:
    print(i,end=" ")
```

Python List Built-in functions

Python provides the following built-in functions, which can be used with the lists.

<u>Method</u>	<u>Description</u>
<code>append()</code>	Adds an element at the end of the list
<code>clear()</code>	Removes all the elements from the list
<code>copy()</code>	Returns a copy of the list
<code>count()</code>	Returns the number of elements with the specified value
<code>extend()</code>	Add the elements of a list (or any iterable), to the end of the current list
<code>index()</code>	Returns the index of the first element with the specified value
<code>insert()</code>	Adds an element at the specified position
<code>pop()</code>	Removes the element at the specified position
<code>remove()</code>	Removes the first item with the specified value
<code>reverse()</code>	Reverses the order of the list
<code>sort()</code>	Sorts the list

append()

Used for appending and adding elements to List. It is used to add elements to the last position of the List in Python.

Syntax:

list.append (element)

Adds List Element as value of List.

```
List = ['Mathematics', 'chemistry', 1997, 2000]
```

```
List.append(20544)
```

```
print(List)
```

Output:

```
['Mathematics', 'chemistry', 1997, 2000, 20544]
```

insert()

Inserts an element at the specified position.

Syntax:

list.insert(<position, element)

Note: Position mentioned should be within the range of List, as in this case between 0 and 4, otherwise would throw IndexError.

```
List = ['Mathematics', 'chemistry', 1997, 2000]
```

```
# Insert at index 2 value 10087
```

```
List.insert(2,10087)
```

```
print(List)
```

Output:

```
['Mathematics', 'chemistry', 10087, 1997, 2000, 20544]
```

extend()

Adds contents to List2 to the end of List1.

Syntax

List1.extend(List2)

```
List1 = [1, 2, 3]
```

```
List2 = [2, 3, 4, 5]
```

```
# Add List2 to List1
```

```
List1.extend(List2)
```

```
print(List1)
```

```
# Add List1 to List2 now
```

```
List2.extend(List1)
```

```
print(List2)
```

Output:

```
[1, 2, 3, 2, 3, 4, 5]
```

```
[2, 3, 4, 5, 1, 2, 3, 2, 3, 4, 5]
```

Other functions of List

sum()

Calculates the sum of all the elements of the List.

Syntax

```
sum(List)
```

```
List = [1, 2, 3, 4, 5]
```

```
print(sum(List))
```

Output:

```
15
```

What happens if a numeric value is not used a parameter?

The sum is calculated only for Numeric values, elsewise throws TypeError.

See example:

```
List = ['gfg', 'abc', 3]
```

```
print(sum(List))
```

Output:

Traceback (most recent call last):

```
File "", line 1, in
```

```
sum(List)
```

TypeError: unsupported operand type(s) for +: 'int' and 'str'

count()

Calculates total occurrence of a given element of List.

Syntax:

List.count(element)

```
List = [1, 2, 3, 1, 2, 1, 2, 3, 2, 1]
print(List.count(1))
```

Output:

4

length

Calculates the total length of the List.

Syntax:

len(list_name)

```
List = [1, 2, 3, 1, 2, 1, 2, 3, 2, 1]
print(len(List))
```

Output:

10

index()

Returns the index of the first occurrence. The start and End index are not necessary parameters.

Syntax:

List.index(element[,start[,end]])

```
List = [1, 2, 3, 1, 2, 1, 2, 3, 2, 1]
print(List.index(2))
```

Output:

1

Another example:

```
List = [1, 2, 3, 1, 2, 1, 2, 3, 2, 1]
print(List.index(2,2))
```

Output:

4

min()

Calculates minimum of all the elements of List.

Syntax:

Deletion of List Elements

To Delete one or more elements, i.e. remove an element, many built-in functions can be used, such as `pop()` & `remove()` and keywords such as `del`.

pop()

The index is not a necessary parameter, if not mentioned takes the last index.

Syntax:

```
list.pop([index])
```

Note: Index must be in range of the List, elsewise `IndexErrors` occurs.

```
List = [2.3, 4.445, 3, 5.33, 1.054, 2.5]
```

```
print(List.pop())
```

Output:

2.5

```
List = [2.3, 4.445, 3, 5.33, 1.054, 2.5]
```

```
print(List.pop(0))
```

Output:

2.3

del()

Element to be deleted is mentioned using list name and index.

Syntax:

```
del list.[index]
```

```
List = [2.3, 4.445, 3, 5.33, 1.054, 2.5]
```

```
del List[0]
```

```
print(List)
```

Output:

[4.445, 3, 5.33, 1.054, 2.5]

remove()

Element to be deleted is mentioned using list name and element.

Syntax

list.remove(element)

```
List = [2.3, 4.445, 3, 5.33, 1.054, 2.5]
```

```
List.remove(3)
```

```
print(List)
```

Output:

[2.3, 4.445, 5.33, 1.054, 2.5]

Pythonlife.in

Introduction to Python String

- A string is just a sequence of characters.
- It is among the most popular data types in Python.
- It can be created simply by enclosing characters in quotes.
- Python provides a rich set of operators, functions, and methods for working with strings which helps to access and extract portions of strings and also manipulate and modify string data.

What is a string in python?

- Technically speaking, an immutable sequence of data is known as a string in Python.
- In simple words, as discussed in the case of a crossword, a Python string is nothing but an array of characters, but a computer does not understand characters.
- It only understands the language of 0's and 1's.
- So these characters are converted to a number so that the computer can understand it clearly.
- This conversion is known as encoding. ASCII and Unicode are some of the popular encodings used.
- To put it simply, Python strings are a sequence of Unicode characters.

Creating a String in Python

- A string in Python can be easily created using single, double or even triple quotes. The characters that form the string are enclosed within any of these quotes.
- Note- Triple quotes are generally used when we are working with multiline strings in Python and docstring in Python.

Reassigning Strings in Python

- Since strings in Python are immutable, we can update the values of strings simply by reassigning these strings with new values.

The strings cannot be partially replaced, so it is always replaced completely with a new string.

- Let's see how that's done –

```
message="Welcome to PythonLife"  
print(message)  
message="Hello World" #reassigning a new string  
print(message)
```

- The output of the above snippet will be –

```
Welcome to PythonLife  
Hello World
```

From the output, it is evident that the string variable message was reassigned with a new string, which gets printed to the console in the last line.

How to access characters in a python string?

As we know, a string is made up of a sequence of characters. Thus, in Python, we have some techniques to reference or index a particular character in a string.

The two primary ways of accessing characters of a string in Python are-

1. Indexing –

The length of a string can be easily calculated using the `len()` function . The length of a string provides us with a range of the string. In python, we can perform indexing on any string as long as it is within the range. The index is always referred as to be an integer. Using any other data type would give rise to a `TypeError`.

Indexing can either be positive or negative. Positive indexing starts from 0 and starts indexing from the beginning of the string. On the other hand, negative indexing starts from -1, and indexing starts from the end of the string.

2. Slicing –

Unlike indexing, slicing returns a range of characters using the slicing syntax.

The syntax looks like –

`'start[start_index: end_index]'`

While performing slicing operations, one thing to remember is that the start index value is always included while that of the last index is always excluded.

Slicing can also be negative or positive, just like indexing in Python!

How to delete a string?

- We have established that strings are immutable. In simple words, it means that once a string is assigned, we cannot make any changes to it. We cannot remove or delete any character of the string.
- What we can do is delete the Python string entirely. We can do so by using the del keyword.

Example Program of String Operators in Python

```
message1="Hello World!"  
message2="Welcome to PythonLife"  
print(message1+message2) # + operator  
print(message1*3) # * operator  
print(message1[6]) # [] operator  
print(message2[0:7]) # [:] operator  
str1="Hello"  
if str1 in message1:# in operator  
    print("It is present!")  
if str1 not in message2:# not in operator  
    print("It is not present!")
```

Output –

Hello World!Welcome to PythonLife

Hello World!Hello World!Hello World!

w

Welcome

It is present!

It is not present!

String Manipulation

To manipulate strings, we can use some of Python's built-in methods.

Creation

```
word = "Hello World"
```

```
>>> print word
```

```
Hello World
```

Accessing

Use [] to access characters in a string

```
word = "Hello World"
```

```
letter=word[0]
```

```
>>> print letter
```

```
H
```

Length

```
word = "Hello World"
```

```
>>> len(word)
```

```
11
```

Finding

```
word = "Hello World">>> print word.count('l') # count how many times l is in the string
```

```
3
```

```
>>> print word.find("H") # find the word H in the string
```

```
0
```

```
>>> print word.index("World") # find the letters World in the string
```


6

Count

```
s = "Count, the number of spaces"
```

```
>>> print s.count(' ')
```

8

Slicing

Use [# : #] to get set of letter

Keep in mind that python, as many other languages, starts to count from 0!!

```
word = "Hello World"
```

```
print word[0] #get one char of the word
```

```
print word[0:1] #get one char of the word (same as above)
```

```
print word[0:3] #get the first three char
```

```
print word[:3] #get the first three char
```

```
print word[-3:] #get the last three char
```

```
print word[3:] #get all but the three first char
```

```
print word[:-3] #get all but the three last character
```

```
word = "Hello World"
```

```
word[start:end] # items start through end-1
```

```
word[start:] # items start through the rest of the list
```

```
word[:end] # items from the beginning through end-1
```

```
word[:] # a copy of the whole list
```

Split Strings

```
word = "Hello World"
```

```
>>> word.split(' ') # Split on whitespace
```

```
['Hello', 'World']
```

Startswith / Endswith

```
word = "hello world"
```

```
>>> word.startswith("H")
```

```
True
```

```
>>> word.endswith("d")
```

True

```
>>> word.endswith("w")
```

False

Repeat Strings

```
print "." * 10 # prints ten dots
```

```
>>> print "." * 10
```

.....

Replacing

```
word = "Hello World"
```

```
>>> word.replace("Hello", "Goodbye")
```

```
'Goodbye World'
```

Changing Upper and Lower Case Strings

```
string = "Hello World"
```

```
>>> print string.upper()
```

```
HELLO WORLD
```

```
>>> print string.lower()
```

```
hello world
```

```
>>> print string.title()
```

```
Hello World
```

```
>>> print string.capitalize()
```

```
Hello world
```

```
>>> print string.swapcase()
```

```
hELLO wORLD
```

Reversing

```
string = "Hello World"
```

```
>>> print ''.join(reversed(string))
```

```
d l r o W o l l e H
```

Strip

Python strings have the `strip()`, `lstrip()`, `rstrip()` methods for removing

any character from both ends of a string.

If the characters to be removed are not specified then white-space will be removed

```
word = "Hello World"
```

Strip off newline characters from end of the string

```
>>> print word.strip('
```

```
')
```

```
Hello World
```

`strip()` #removes from both ends

`lstrip()` #removes leading characters (Left-strip)

`rstrip()` #removes trailing characters (Right-strip)

```
>>> word = " xyz "
```

```
>>> print word
```

```
xyz
```

```
>>> print word.strip()
```

```
xyz
```

```
>>> print word.lstrip()
```

```
xyz
```

```
>>> print word.rstrip()
```

xyz

Concatenation

To concatenate strings in Python use the “+” operator.

```
"Hello " + "World" # = "Hello World"
```

```
"Hello " + "World" + "!"# = "Hello World!"
```

Join

```
>>> print ":".join(word) # #add a : between every char
```

```
H:e:l:l:o: :W:o:r:l:d
```

```
>>> print " ".join(word) # add a whitespace between every char
```

```
H e l l o W o r l d
```

Extra information:

`lower()` Converts a string into lower case

`rstrip()` Returns a left trim version of the string

`replace()` Returns a string where a specified value is replaced with a specified value

- `rstrip()` Returns a right trim version of the string
- `split()` Splits the string at the specified separator, and returns a list
- `startswith()` Returns true if the string starts with the specified value
- `strip()` Returns a trimmed version of the string
- `title()` Converts the first character of each word to upper case
- `upper()` Converts a string into upper case
- `count()` Returns the number of times a specified value occurs in a string
- `endswith()` Returns true if the string ends with the specified value
- `find()` Searches the string for a specified value and returns the position of where it was found
- `format()` Formats specified values in a string
- `index()` Searches the string for a specified value and returns the position of where it was found
- `isalnum()` Returns True if all characters in the string are alphanumeric
- `isalpha()` Returns True if all characters in the string are in the alphabet
- `join()` Converts the elements of an iterable into a string

Python Dictionary

Python dictionary is an unordered collection of items. Each item of a dictionary has a key/value pair.

Dictionaries are optimized to retrieve values when the key is known.

Creating Python Dictionary

Creating a dictionary is as simple as placing items inside curly braces { } separated by commas.

An item has a key and a corresponding value that is expressed as a pair (key: value).

While the values can be of any data type and can repeat, keys must be of immutable type (string, number or tuple with immutable elements) and must be unique.

```
# empty dictionary
my_dict = {}
# dictionary with integer keys
my_dict = {1: 'apple', 2: 'ball'}
# dictionary with mixed keys
my_dict = {'name': 'John', 1: [2, 4, 3]}
# using dict()
my_dict = dict({1:'apple', 2:'ball'})
# from sequence having each item as a pair
my_dict = dict([(1,'apple'), (2,'ball')])
```

As you can see from above, we can also create a dictionary using the built-in dict() function.

Accessing Elements from Dictionary

While indexing is used with other data types to access values, a dictionary uses keys.

Keys can be used either inside square brackets [] or with the get() method.

If we use the square brackets [], KeyError is raised in case a key is not found in the dictionary. On the other hand, the get() method returns None if the key is not found.

```
# get vs [] for retrieving elements
my_dict = {'name': 'Jack', 'age': 26}
# Output: Jack
print(my_dict['name'])
# Output: 26
print(my_dict.get('age'))
# Trying to access keys which doesn't exist throws error
# Output None
print(my_dict.get('address'))
# KeyError
print(my_dict['address'])
```

Changing and Adding Dictionary elements

Dictionaries are mutable. We can add new items or change the value of existing items using an assignment operator.

If the key is already present, then the existing value gets updated. In case the key is not present, a new (key: value) pair is added to the dictionary.

```
# Changing and adding Dictionary Elements
my_dict = {'name': 'Jack', 'age': 26}
# update value
```

```
my_dict['age'] = 27
#Output: {'age': 27, 'name': 'Jack'}
print(my_dict)
# add item
my_dict['address'] = 'Downtown'
# Output: {'address': 'Downtown', 'age': 27, 'name': 'Jack'}
```

Removing elements from Dictionary

We can remove a particular item in a dictionary by using the `pop()` method. This method removes an item with the provided key and returns the value

The `popitem()` method can be used to remove and return an arbitrary (key,value) item pair from the dictionary. All the items can be removed at once, using the `clear()` method.

We can also use the `del()` keyword to remove individual items or the entire dictionary itself.

```
# Removing elements from a dictionary
# create a dictionary
squares = {1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
# remove a particular item, returns its value
# Output: 16
print(squares.pop(4))
# Output: {1: 1, 2: 4, 3: 9, 5: 25}
print(squares)
# remove an arbitrary item, return (key,value)
# Output: (5, 25)
print(squares.popitem())
# Output: {1: 1, 2: 4, 3: 9}
print(squares)
# remove all items
squares.clear()
# Output: {}
print(squares)
# delete the dictionary itself
del squares
# Throws Error
print(squares)
```

Python Dictionary Methods

Methods that are available with a dictionary are tabulated below. Some of them have already been used in the above examples.

Method	Description
<code>clear()</code>	Removes all items from the dictionary.
<code>copy()</code>	Returns a shallow copy of the dictionary.
<code>get(key[,d])</code>	Returns the value of the key. If the key does not exist, returns d
<code>items()</code>	Return a new object of the dictionary's items in (key, value) format.
<code>keys()</code>	Returns a new object of the dictionary's keys.
<code>update([other])</code>	Updates the dictionary with the key/value pairs from other, overwriting existing keys.
<code>values()</code>	Returns a new object of the dictionary's values

Python Dictionary Comprehension

Dictionary comprehension is an elegant and concise way to create a new dictionary from an iterable in Python.

Dictionary comprehension consists of an expression pair (key: value) followed by a for statement inside curly braces { }

Here is an example to make a dictionary with each item being a pair of a number and its square.

```
# Dictionary Comprehension
squares = {x: x*x for x in range(6)}
print(squares)
```

Output

```
{0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

A dictionary comprehension can optionally contain more for or if statements.

An optional if statement can filter out items to form the new dictionary.

Here are some examples to make a dictionary with only odd items.

```
# Dictionary Comprehension with if conditional
odd_squares = {x: x*x for x in range(11) if x % 2 == 1}
print(odd_squares)
```

Output

```
{1: 1, 3: 9, 5: 25, 7: 49, 9: 81}
```

Other Dictionary Operations

Dictionary Membership Test

We can test if a key is in a dictionary or not using the keyword `in`. Notice that the membership test is only for the keys and not for the values.

```
# Membership Test for Dictionary Keys
squares = {1: 1, 3: 9, 5: 25, 7: 49, 9: 81}
# Output: True
print(1 in squares)
# Output: True
print(2 not in squares)
# membership tests for key only not value
# Output: False
```

```
print(49 in squares)
```

Iterating Through a Dictionary

We can iterate through each key in a dictionary using a for loop

```
# Iterating through a Dictionary
squares = {1: 1, 3: 9, 5: 25, 7: 49, 9: 81}
for i in squares:
    print(squares[i])
```

Output

```
1
9
25
49
81
```

Python Tuple

A tuple in Python is similar to a list. The difference between the two is that we cannot change the elements of a tuple once it is assigned whereas we can change the elements of a list.

Creating a Tuple

A tuple is created by placing all the items (elements) inside parentheses(), separated by commas. The parentheses are optional, however, it is a good practice to use them.

A tuple can have any number of items and they may be of different types (integer, float, list, string, etc.).

```
my_tuple = ()  
print(my_tuple)  
my_tuple = (1, 2, 3)  
print(my_tuple)  
my_tuple = (1, "Hello", 3.4)  
print(my_tuple)  
my_tuple = ("mouse", [8, 4, 6], (1, 2, 3))  
print(my_tuple)
```

Access Tuple Elements

There are various ways in which we can access the elements of a tuple.

1. Indexing

We can use the index operator [] to access an item in a tuple, where the index starts from 0. So, a tuple having 6 elements will have indices from 0 to 5. Trying to access an index outside of the tuple index range(6,7,... in this example) will raise an `IndexError`

The index must be an integer, so we cannot use float or other types. This will result in `TypeError`.

Likewise, nested tuples are accessed using nested indexing, as shown in the example below.

```
my_tuple = ('p','e','r','m','i','t')
print(my_tuple[0]) # 'p'
print(my_tuple[5]) # 't'

n_tuple = ("mouse", [8, 4, 6], (1, 2, 3))

# nested index
print(n_tuple[0][3])      # 's'
print(n_tuple[1][1])      # 4
```

2. Negative Indexing

Python allows negative indexing for its sequences.

The index of -1 refers to the last item, -2 to the second last item and so on.

```
my_tuple = ('p', 'e', 'r', 'm', 'i', 't')
print(my_tuple[-1])    # 't'
print(my_tuple[-6])    # 'p'
```

In-built Functions for Tuple:

Function	Description
cmp(tuple1,tuple2)	Compares elements of two different tuples
len(tuple)	Returns the length of the tuple
max(tuple)	Returns the element with max value from the tuple
min(tuple)	Returns the element with min value from the tuple
tuple(seq)	Returns a tuple, by converting a list to a tuple. Takes a list in the parameter

3. Slicing

We can access a range of items in a tuple by using the slicing operator colon :

```
my_tuple = ('p','r','o','g','r','a','m','i','z')
```

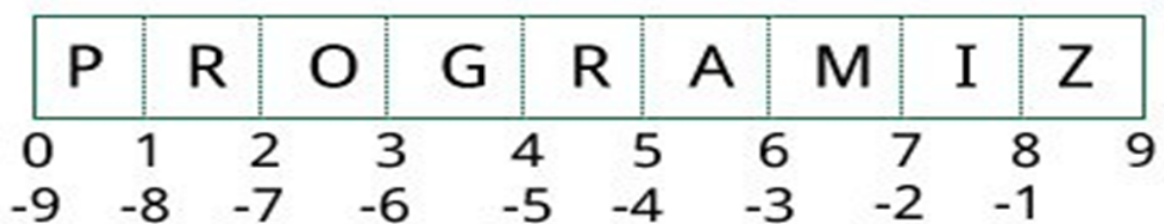
```
print(my_tuple[1:4])
```

```
print(my_tuple[:-7])
```

```
print(my_tuple[7:])
```

```
print(my_tuple[:])
```

Slicing can be best visualized by considering the index to be between the elements as shown below. So if we want to access a range, we need the index that will slice the portion from the tuple.



Changing a Tuple

Unlike lists, tuples are immutable.

This means that elements of a tuple cannot be changed once they have been assigned. But, if the element is itself a mutable data type like a list, its nested items can be changed.

We can also assign a tuple to different values (reassignment).

```
my_tuple = (4, 2, 3, [6, 5])  
my_tuple[3][0] = 9      # Output: (4, 2, 3, [9, 5])  
print(my_tuple)  
my_tuple = ('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')  
print(my_tuple)
```

We can use + operator to combine two tuples. This is called concatenation.

We can also repeat the elements in a tuple for a given number of times using the * operator.

Both + and * operations result in a new tuple.

```
# Concatenation  
# Output: (1, 2, 3, 4, 5, 6)  
print((1, 2, 3) + (4, 5, 6))  
# Repeat  
# Output: ('Repeat', 'Repeat', 'Repeat')  
print(("Repeat",) * 3)
```

Deleting a Tuple

As discussed above, we cannot change the elements in a tuple. It means that we cannot delete or remove items from a tuple.

Deleting a tuple entirely, however, is possible using the keyword `del`.

```
# Deleting tuples

my_tuple = ('p', 'r', 'o', 'g', 'r', 'a', 'm', 'i', 'z')

# can't delete items

# TypeError: 'tuple' object doesn't support item deletion

# del my_tuple[3]

# Can delete an entire tuple

del my_tuple

# NameError: name 'my_tuple' is not defined

print(my_tuple)
```

Tuple Methods

Methods that add items or remove items are not available with tuple. Only the following two methods are available.

Some examples of Python tuple methods:

```
my_tuple = ('a', 'p', 'p', 'l', 'e')

print(my_tuple.count('p')) # Output: 2

print(my_tuple.index('l')) # Output: 3
```

Other Tuple Operations

1. Tuple Membership Test

We can test if an item exists in a tuple or not, using the keyword `in`

```
# Membership test in tuple
my_tuple = ('a', 'p', 'p', 'l', 'e')

# In operation
print('a' in my_tuple)
print('b' in my_tuple)

# Not in operation
print('g' not in my_tuple)
```

2. Iterating Through a Tuple

We can use a for loop to iterate through each item in a tuple.

```
# Using a for loop to iterate through a tuple
for name in ('John', 'Kate'):
    print("Hello", name)
```

Advantages of Tuple

- Tuples being immutable, turns out to be a write-protected collection. Tuples can be of advantage when we want to store some secure read-only data that we cannot afford to be changed throughout our code.
- Tuples can store data of multiple data types, which makes them a heterogeneous collection.
- Tuple being a read only collection, has a faster iteration. (As they are stored in a single block of memory, and don't have extra space for storing objects, they have a constant set of values)

Disadvantages of Tuple of Tuple

- Tuple's being write protected, is an advantage but also a disadvantage as it cannot be used when we want to add or delete a specific element. Hence has a limited use case.
- Syntactically less readable as tuples can be created by either adding parentheses or by not providing them in case we have more than one element. But not using parentheses in the case of one element, will not create a tuple and hence a trailing comma in such case is required. This can make code readability a bit complex for some.
- As tuple is a class, it's stored on the heap and is overhead on the garbage collector.

Python Sets

A set is an unordered collection of items. Every set element is unique (no duplicates) and must be immutable (cannot be changed).

However, a set itself is mutable. We can add or remove items from it.

Sets can also be used to perform mathematical set operations like union, intersection, symmetric difference, etc.

Creating Python Sets

A set is created by placing all the items (elements) inside curly braces { }, separated by comma, or by using the built-in set() function.

It can have any number of items and they may be of different types (integer, float, tuple, string etc.). But a set cannot have mutable elements like lists, sets or dictionaries as its elements.

```
# Different types of sets in Python
```

```
# set of integers
```

```
my_set = {1, 2, 3}
```

```
print(my_set)
```

```
# set of mixed datatypes
```

```
my_set = {1.0, "Hello", (1, 2, 3)}
```

```
print(my_set)
```

Creating an empty set is a bit tricky.

Empty curly braces { } will make an empty dictionary in Python. To make a set without any elements, we use the set () function without any argument.

```
# Distinguish set and dictionary while creating empty set

# initialize a with {}

a = {}

# check data type of a

print(type(a))

# initialize a with set()

a = set()

# check data type of a

print(type(a))
```

Modifying a set in Python

Sets are mutable. However, since they are unordered, indexing has no meaning.

We cannot access or change an element of a set using indexing or slicing. Set data type does not support it.

We can add a single element using the add () method, and multiple elements using the update () method. The update () method can take tuples, lists, strings or other sets as its argument. In all cases, duplicates are avoided.

```
# initialize my_set

my_set = {1, 3}

print(my_set)

# my_set[0]

# if you uncomment the above line

# you will get an error

# TypeError: 'set' object does not support indexing

# add an element

# Output: {1, 2, 3}

my_set.add(2)

print(my_set)

# add multiple elements
```

```
# Output: {1, 2, 3, 4}
my_set.update([2, 3, 4])
print(my_set)

# add list and set

# Output: {1, 2, 3, 4, 5, 6, 8}
my_set.update([4, 5], {1, 6, 8})
print(my_set)
```

Removing elements from a set

A particular item can be removed from a set using the methods `discard ( )` and `remove ( )`

The only difference between the two is that the `discard ( )` function leaves a set unchanged if the element is not present in the set. On the other hand, the `remove ( )` function will raise an error in such a condition (if element is not present in the set).

The following example will illustrate this.

```
# Difference between discard() and remove()

# initialize my_set
my_set = {1, 3, 4, 5, 6}
print(my_set)

# discard an element

# Output: {1, 3, 5, 6}
my_set.discard(4)
print(my_set)

# remove an element

# Output: {1, 3, 5}
my_set.remove(6)
print(my_set)

# discard an element

# not present in my_set
```

```
# Output: {1, 3, 5}
```

```
my_set.discard(2)
```

```
print(my_set)
```

```
# remove an element
```

```
# not present in my_set
```

```
# you will get an error.
```

```
# Output: KeyError
```

```
my_set.remove(2)
```

Similarly, we can remove and return an item using the pop () method.

Since set is an unordered data type, there is no way of determining which item will be popped. It is completely arbitrary.

We can also remove all the items from a set using the clear () method

```
# initialize my_set
```

```
# Output: set of unique elements
```

```
my_set = set("HelloWorld")
```

```
print(my_set)
```

```
# pop an element
```

```
# Output: random element
```

```
print(my_set.pop())
```

```
# pop another element
```

```
my_set.pop()
```

```
print(my_set)
```

```
# clear my_set
```

```
# Output: set()
```

```
my_set.clear()
```

```
print(my_set)
```


Python Set Operations

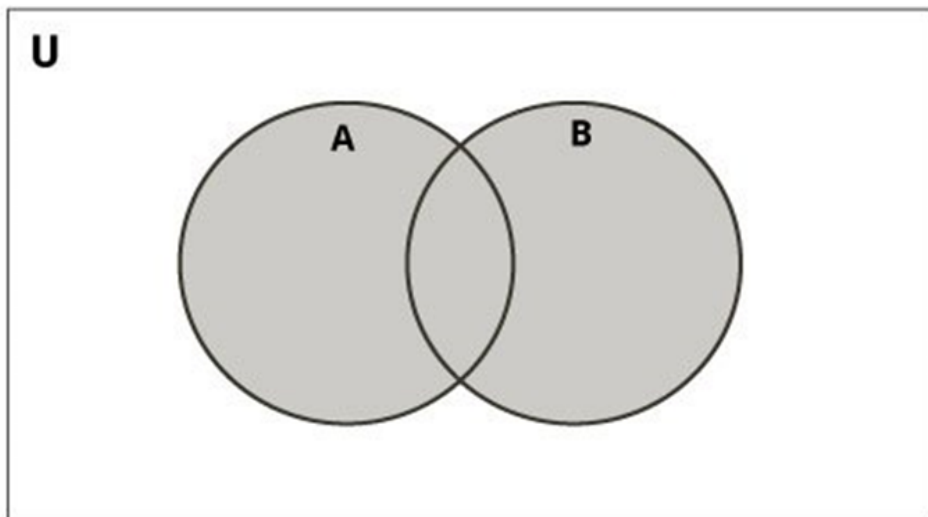
Sets can be used to carry out mathematical set operations like union, intersection, difference and symmetric difference. We can do this with operators or methods.

Let us consider the following two sets for the following operations.

$A = \{1, 2, 3, 4, 5\}$

$B = \{4, 5, 6, 7, 8\}$

Set Union



Set Union in Python

Union of A and B is a set of all elements from both sets.

Union is performed using `|` operator. Same can be accomplished using the union method.

```
# Set union method
```

```
# initialize A and B
```

```
A = {1, 2, 3, 4, 5}
```

```
B = {4, 5, 6, 7, 8}
```

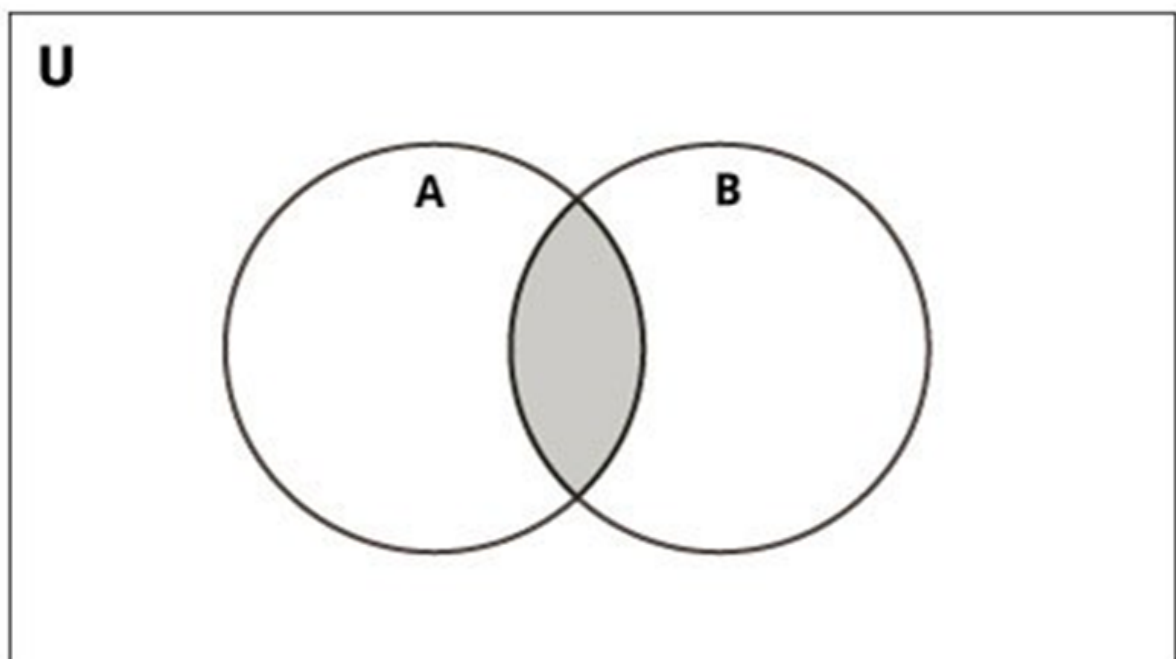
```
# use | operator
```

```
# Output: {1, 2, 3, 4, 5, 6, 7, 8}
```

```
print(A | B)
```

Output

```
{1, 2, 3, 4, 5, 6, 7, 8}
```

Set Intersection

Set Intersection in Python

Intersection of A and B is a set of elements that are common in both the sets

Intersection is performed using & operator. Same can be accomplished using the Intersection () method

```
# Intersection of sets
```

```
# initialize A and B
```

```
A = {1, 2, 3, 4, 5}
```

```
B = {4, 5, 6, 7, 8}
```

```
# use & operator
```

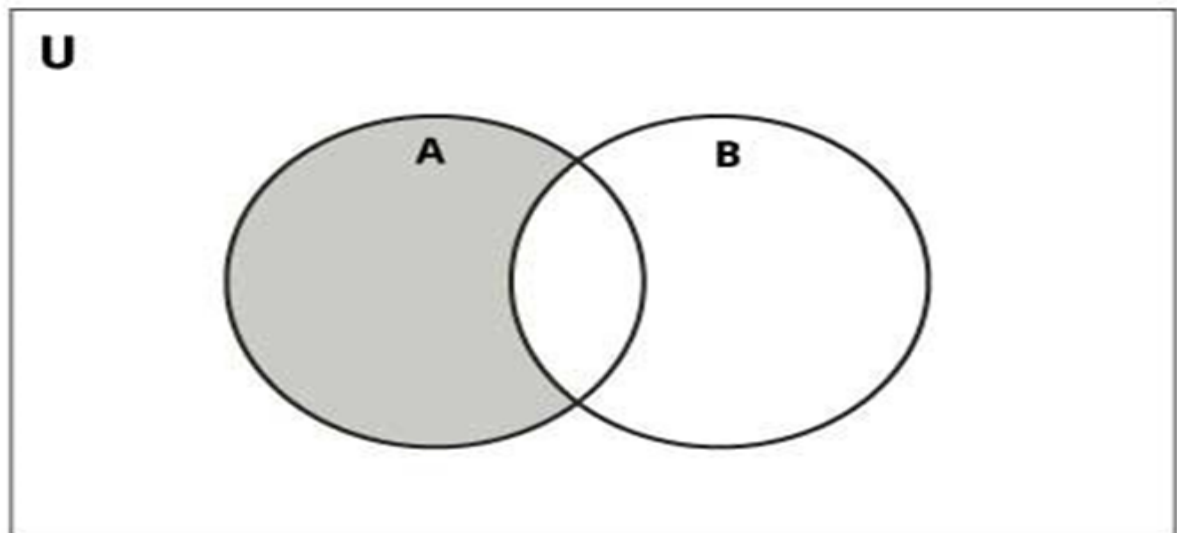
```
# Output: {4, 5}
```

```
print(A & B)
```

Output

```
{ 4, 5 }
```

Set Difference



Set Difference in Python

Difference of the set B from set A ($A-B$) is a set of elements that are only in A but not in B .

Similarly, $B-A$ is a set of elements in B but not in A .

Difference is performed using $-$ operator. Same can be accomplished using the difference () method.

```
# Difference of two sets
```

```
# initialize A and B
```

```
A = {1, 2, 3, 4, 5}
```

```
B = {4, 5, 6, 7, 8}
```

```
# use - operator on A
```

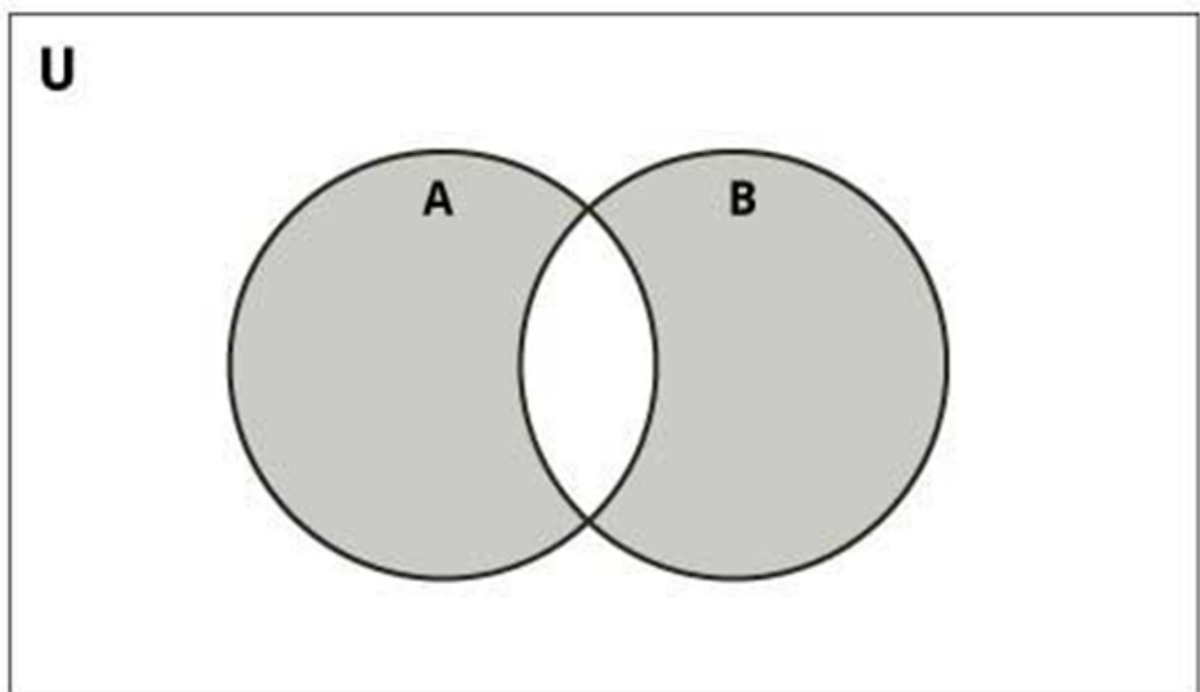
```
# Output: {1, 2, 3}
```

```
print(A - B)
```

Output

```
{1, 2, 3}
```

Set Symmetric Difference



Set Symmetric Difference in python

Symmetric Difference of A and B is a set of elements in A and B but not in both (excluding the intersection).

Symmetric difference is performed using $\wedge$ operator. Same can be accomplished using the method `symmetric_difference()`

```
# Symmetric difference of two sets
```

```
# initialize A and B
```

```
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}
# use ^ operator
# Output: {1, 2, 3, 6, 7, 8}
print(A ^ B)
```

Output

```
{1, 2, 3, 6, 7, 8}
```

Other Python Set Methods

There are many set methods, some of which we have already used above. Here is a list of all the methods that are available with the set objects:

Other Set Operations

Set Membership Test

We can test if an item exists in a set or not, using the `in` keyword

```
# in keyword in a set
# initialize my_set
my_set = set("apple")
# check if 'a' is present
# Output: True
print('a' in my_set)
# check if 'p' is present
# Output: False
print('p' not in my_set)
```

Iterating Through a Set

We can iterate through each item in a set using a for loop

for letter in set ("apple"):

```
    print(letter)
```

Output

a

p

l

e

Python Frozenset

Frozenset is a new class that has the characteristics of a set, but its elements cannot be changed once assigned. While tuples are immutable lists, frozensets are immutable sets.

Sets being mutable are unhashable, so they can't be used as dictionary keys. On the other hand, frozensets are hashable and can be used as keys to a dictionary.

Frozensets can be created using the `frozenset()` function.

This data type supports methods like: `copy()`, `difference()`, `intersection()`, `isdisjoint()`, `issubset()`, `issuperset()`, `symmetric_difference()`, and `union()`. Being immutable, it does not have methods that add or remove elements.

Pythonlife.in

What is a function in Python?

In Python, a function is a group of related statements that performs a specific task.

Functions help break our program into smaller and modular chunks. As our program grows larger and larger, functions make it more organized and manageable.

Furthermore, it avoids repetition and makes the code reusable.

Syntax of Function

```
def dunction_name(parameters)
    “ “ “ doc string ”””
    statement(s)
```

Above shown is a function definition that consists of the following components.

- 1.Keyword `def` that marks the start of the function header.
- 2.A function name to uniquely identify the function. Function naming follows the same rules of writing identifiers in Python.
- 3.Parameters (arguments) through which we pass values to a function. They are optional.
4. A colon (`:`) to mark the end of the function header.

5.Optional documentation string (docstring) to describe what the function does.

6.One or more valid python statements that make up the function body. Statements must have the same indentation level (usually 4 spaces).

7.An optional return statement to return a value from the function.

Example of a function

```
def greet(name):  
    """  
    This function greets to  
    the person passed in as  
    a parameter  
    """  
    print("Hello, " + name + ". Good morning!")
```

How to call a function in python?

Once we have defined a function, we can call it from another function, program, or even the Python prompt. To call a function we simply type the function name with appropriate parameters.

```
greet('Paul')
```

Hello, Paul. Good morning!

Try running the above code in the Python program with the function definition to see the output.

```
def greet(name):  
    """ This function greets to
```

the person passed in as

a parameter """

```
print("Hello, " + name + ". Good morning!")
```

```
greet('Paul')
```

Output

Hello, Paul. Good morning!

Note: In python, the function definition should always be present before the function call. Otherwise, we will get an error. For example,

function call

```
greet('Paul')
```

function definition

```
def greet(name):
```

```
    """
```

```
    This function greets to
```

```
    the person passed in as
```

```
    a parameter
```

```
    """
```

```
    print("Hello, " + name + ". Good morning!")
```

Error: name 'greet' is not defined

Docstrings

The first string after the function header is called the docstring and is short for documentation string. It is briefly used to explain what a function does.

Although optional, documentation is a good programming practice. Unless you can remember what you had for dinner last week, always document your code.

In the above example, we have a docstring immediately below the function header. We generally use triple quotes so that docstring can extend up to multiple lines. This string is available to us as the `__doc__` attribute of the function.

For example:

Try running the following into the Python shell to see the output.

```
print(greet.__doc__)
```

```
This function greets to  
the person passed in as  
a parameter
```

The return statement

The return statement is used to exit a function and go back to the place from where it was called.

Syntax of return

```
return [expression_list]
```

This statement can contain an expression that gets evaluated and the value is returned. If there is no expression in the statement or the return statement itself is not present inside a function, then the function will return the `None` object.

For example:

```
print(greet("May"))
```

Hello, May. Good morning!

None

Here, None is the returned value since `greet( )` directly prints the name and no return statement is used.

Example of return

```
def absolute_value(num):
```

```
    """This function returns the absolute  
    value of the entered number"""
```

```
    if num >= 0:
```

```
        return num
```

```
    else:
```

```
        return -num
```

```
print(absolute_value(2))
```

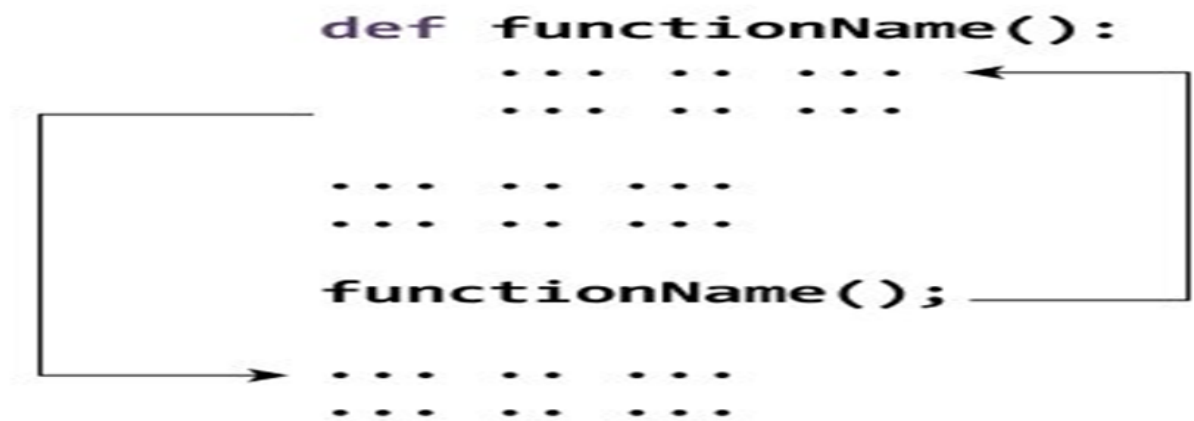
```
print(absolute_value(-4))
```

Output

2

4

How Function works in Python?



Working of functions in Python

Scope and Lifetime of variables

Scope of a variable is the portion of a program where the variable is recognized. Parameters and variables defined inside a function are not visible from outside the function. Hence, they have a local scope.

The lifetime of a variable is the period throughout which the variable exists in the memory. The lifetime of variables inside a function is as long as the function executes.

They are destroyed once we return from the function. Hence, a function does not remember the value of a variable from its previous calls.

Here is an example to illustrate the scope of a variable inside a function.

```
def my_func():  
    x = 10  
  
    print("Value inside function:",x)  
  
x = 20  
  
my_func()  
  
print("Value outside function:",x)
```

Output

Value inside function: 10

Value outside function: 20

Here, we can see that the value of x is 20 initially. Even though the function my_func() changed the value of x to 10, it did not affect the value outside the function.

This is because the variable x inside the function is different (local to the function) from the one outside. Although they have the same names, they are two different variables with different scopes.

On the other hand, variables outside of the function are visible from inside. They have a global scope.

We can read these values from inside the function but cannot change (write) them. In order to modify the value of variables outside the function, they must be declared as global variables using the keyword global.

Types of Functions

Basically, we can divide functions into the following two types:

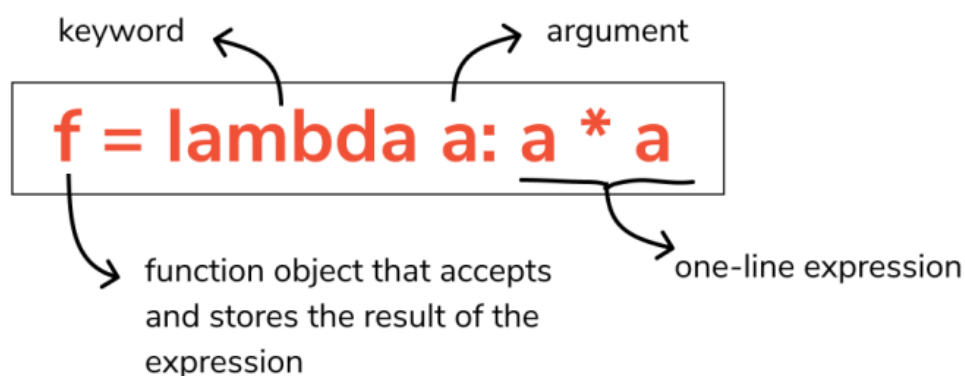
1. Built-in functions - Functions that are built into Python.
2. User-defined functions - Functions defined by the users themselves.

Advance Functions

Lambda Function

A lambda function is a small anonymous function.

A lambda function can take any number of arguments, but can only have one expression.



Map Function

`map()` function returns a map object(which is an iterator) of the results after applying the given function to each item of a given iterable (list, tuple etc.)

Syntax :

`map(fun, iter)`

`fun` : It is a function to which map passes each element of given iterable.

`iter` : It is a iterable which is to be mapped.

```
# Python program to demonstrate working
```

```
# of map.
```

```
# Return double of n
```

```
def addition(n):
```

```
    return n + n
```

```
# We double all numbers using map()
```

```
numbers = (1, 2, 3, 4)
```



```
result = map(addition, numbers)
print(list(result))
```

Filter Function

The `filter()` method filters the given sequence with the help of a function that tests each element in the sequence to be true or not.

syntax:

`filter(function, sequence)`

function: function that tests if each element of a sequence true or not.

sequence: sequence which needs to be filtered, it can be sets, lists, tuples, or containers of any iterators.

```
# function that filters vowels
```

```
def fun(variable):
```

```
    letters = ['a', 'e', 'i', 'o', 'u']
```

```
    if (variable in letters):
```

```
        return True
```

```
    else:
```

```
        return False
```

```
# sequence
```

```
sequence = ['g', 'e', 'e', 'j', 'k', 's', 'p', 'r']
```

```
# using filter function
```

```
filtered = filter(fun, sequence)
```

```
print('The filtered letters are:')
```

```
for s in filtered:
```

```
    print(s)
```

Generator Function

A generator-function is defined like a normal function, but whenever it needs to generate a value, it does so with the `yield` keyword rather than `return`. If the body of a `def` contains `yield`, the function automatically becomes a generator function.

The difference between the `yield` and the `return` statement in Python is:

Return. A function that returns a value is called once. The `return` statement returns a value and exits the function altogether.

Yield. A function that yields values, is called repeatedly. The `yield` statement pauses the execution of a function and returns a value. When called again, the function continues execution from the previous `yield`. A function that yields values is known as a generator.

```
# A generator function that yields 1 for first time,
```

```
# 2 second time and 3 third time
```

```
def simpleGeneratorFun():
```

```
    yield 1
```

```
    yield 2
```

```
    yield 3
```

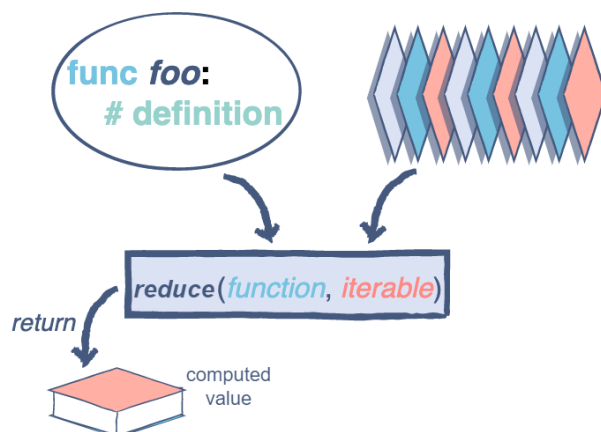
```
# Driver code to check above generator function
```

```
for value in simpleGeneratorFun():
```

```
    print(value)
```

Reduce Function

the `reduce()` function receives two arguments, a function and an iterable. However, it doesn't return another iterable, instead it returns a single value.



```
# python code to demonstrate working of reduce()
```

```
# importing functools for reduce()
```

```
import functools
```

```
# initializing list
```

```
lis = [1, 3, 5, 6, 2]
```

```
# using reduce to compute sum of list
```

```
print("The sum of the list elements is : ", end="")
```

```
print(functools.reduce(lambda a, b: a+b, lis))
```

```
# using reduce to compute maximum element from list
```

```
print("The maximum element of the list is : ", end="")
```

```
print(functools.reduce(lambda a, b: a if a > b else b, lis))
```